Support Vector Machines

 HML – cap 5

 ISLR – cap 9

Support Vector Machines (SVM)

- Pre 1980:
  - Almost all learning methods learned linear decision surfaces.
  - Linear learning methods have nice theoretical properties
- 1980's
  - Decision trees and NNs allowed efficient learning of non- linear decision surfaces
  - Little theoretical basis and all suffer from local minima
- 1990's
  - Efficient learning algorithms for non-linear functions based on computational learning theory developed
  - Nice theoretical properties.

Key Ideas

Two independent developments within last two decades

- New efficient separability of non-linear regions that use “kernel functions” : generalization of ‘similarity’ to new kinds of similarity measures based on dot products
- Use of quadratic optimization problem to avoid ‘local minimum’ issues with neural nets
- The resulting learning algorithm is an optimization algorithm rather than a greedy search

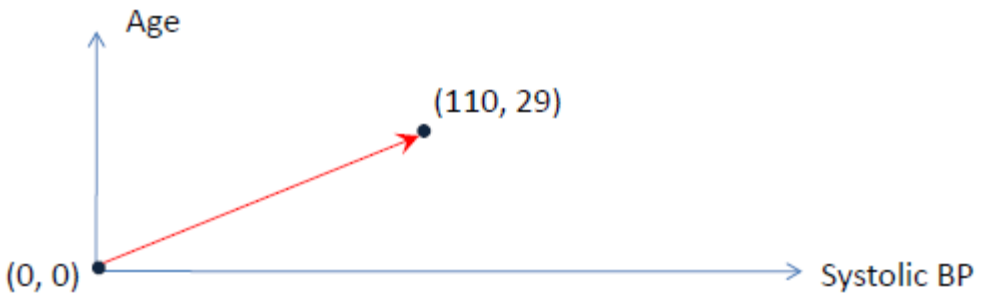
How to represent samples geometrically?

Assume that a sample/patient is described by *n* characteristics ("features" or "variables")

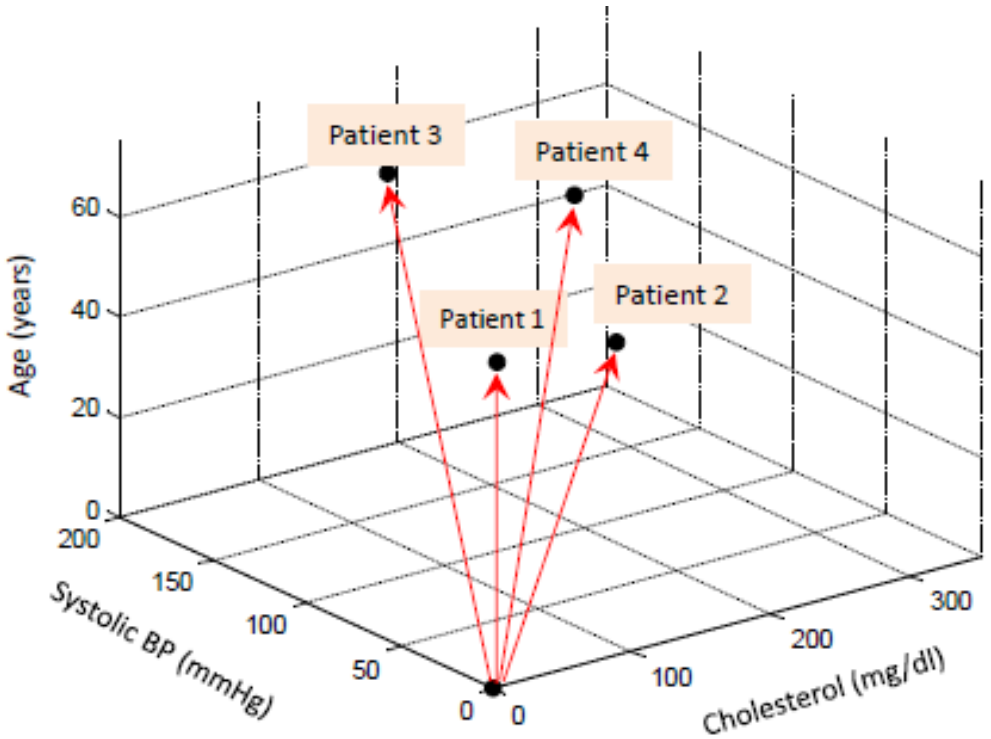
- **Representation:** Every sample/patient is a vector in R^n with tail at point with 0 coordinates and arrow-head at point with the feature values.
- **Example:** Consider a patient described by 2 features:

Systolic BP= 110 and *Age*= 29.

This patient can be represented as a vector in R^2:



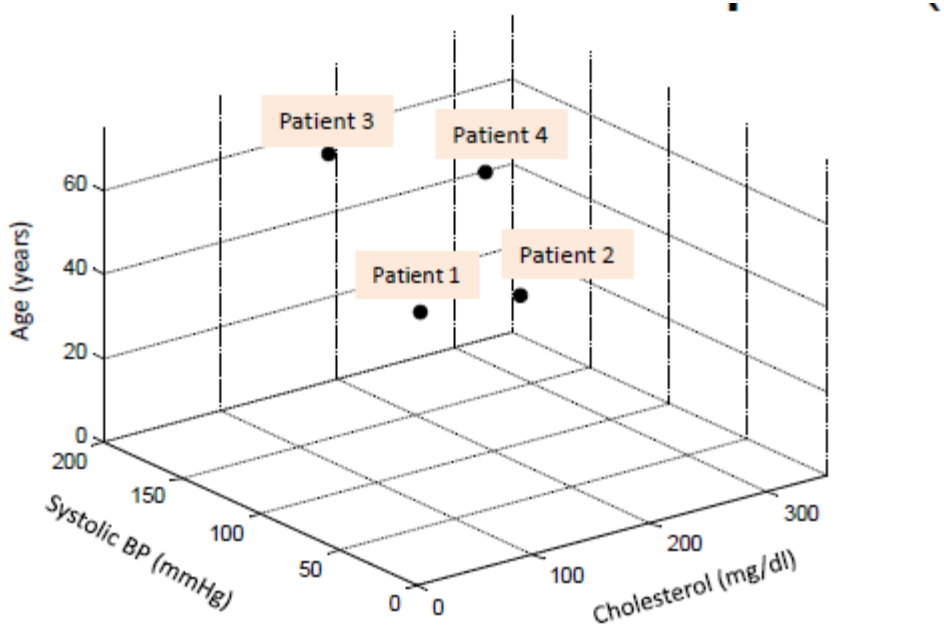
How to represent samples geometrically?



Patient id	Cholesterol (mg/dl)	Systolic BP (mmHg)	Age (years)	Tail of the vector	Arrow-head of the vector
1	150	110	35	(0,0,0)	(150, 110, 35)
2	250	120	30	(0,0,0)	(250, 120, 30)
3	140	160	65	(0,0,0)	(140, 160, 65)
4	300	180	45	(0,0,0)	(300, 180, 45)

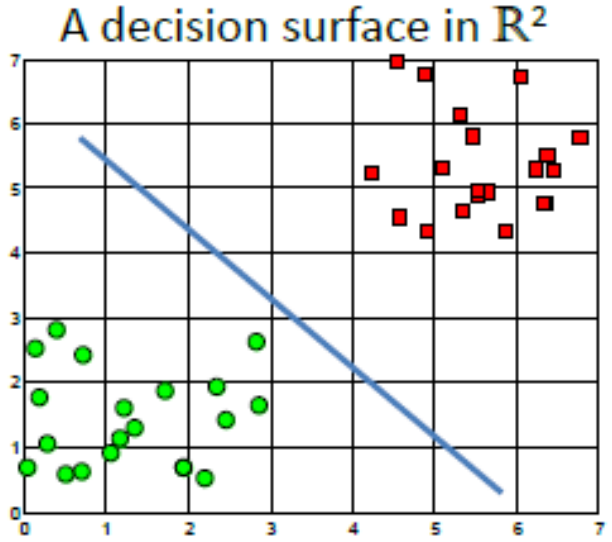
How to represent samples geometrically?

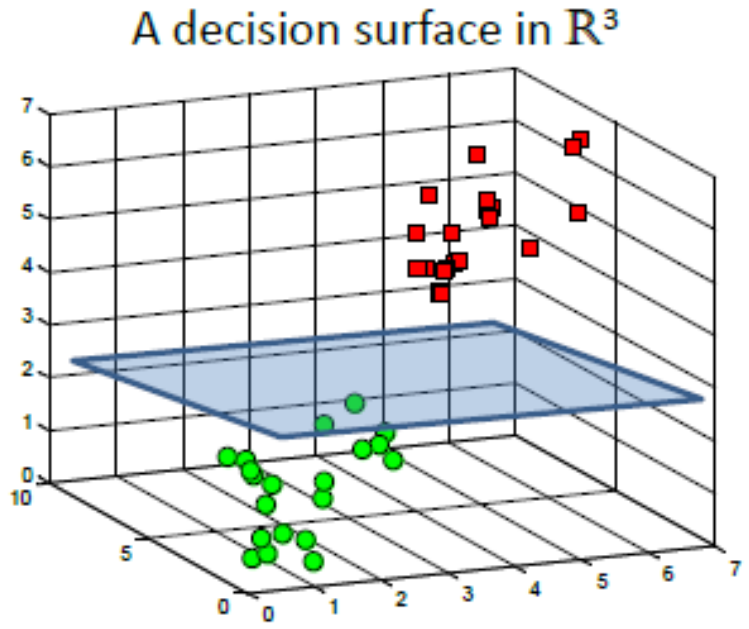
Since we assume that the tail of each vector is at point with 0 coordinates, we will also depict vectors as points (where the arrow-head is pointing).



Purpose of vector representation

Having represented each sample/patient as a vector allows now to geometrically represent the decision surface that separates two groups of samples/patients.





Basic operation on vectors

Multiplication by a scalar

Consider a vector $\vec{a} = (a_1, a_2, \ldots, a_n)$ and scalar c

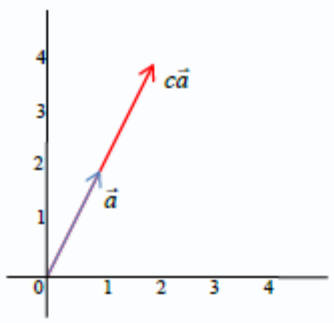
Define: $c\vec{a} = (ca_1, ca_2, \ldots, ca_n)$

When you multiply a vector by a scalar, you "stretch" it in the same or opposite direction depending on whether the scalar is positive or negative.

$\vec{a} = (1,2)$

$c = 2$

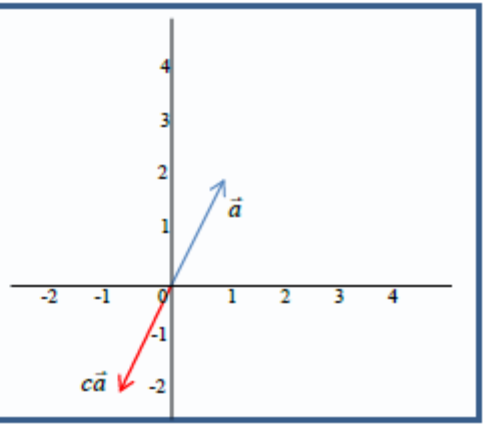
$\vec{c}a = (2,4)$



$\vec{a} = (1,2)$

$c = -1$

$\vec{c}a = (-1,-2)$



Basic operation on vectors

Addition

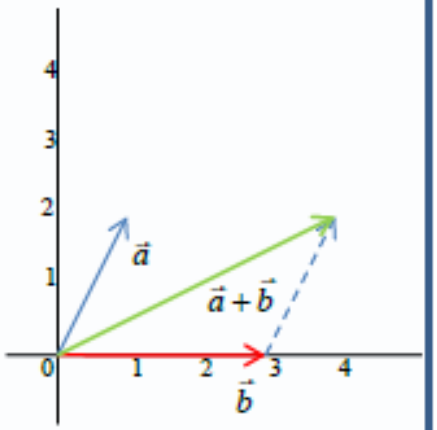
Consider vectors $\vec{a} = (a_1, a_2, \ldots, a_n)$ and $\vec{b} = (b_1, b_2, \ldots, b_n)$

Define: $\vec{a} + \vec{b} = (a_1 + b_1, a_2 + b_2, \ldots, a_n + b_n)$

$\vec{a} = (1,2)$

$\vec{b} = (3,0)$

$\vec{a} + \vec{b} = (4,2)$



Basic operation on vectors

Euclidian length or L2-norm

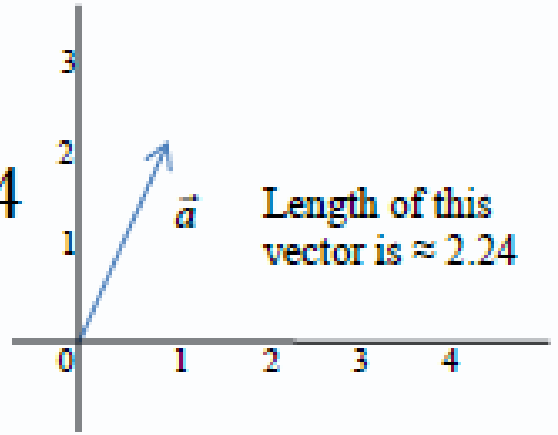
Consider a vector $\vec{a} = (a_1, a_2, \ldots, a_n)$

Define the L2-norm: $\|\vec{a}\|_2 = \sqrt{a_1^2 + a_2^2 + \ldots + a_n^2}$

We often denote the L2-norm without subscript, i.e. $\|\vec{a}\|$

$\vec{a} = (1,2)$

$\|\vec{a}\|_2 = \sqrt{5} \approx 2.24$



L2-norm is a typical way to measure length of a vector; other methods to measure length also exist.

Basic operation on vectors

Dot product

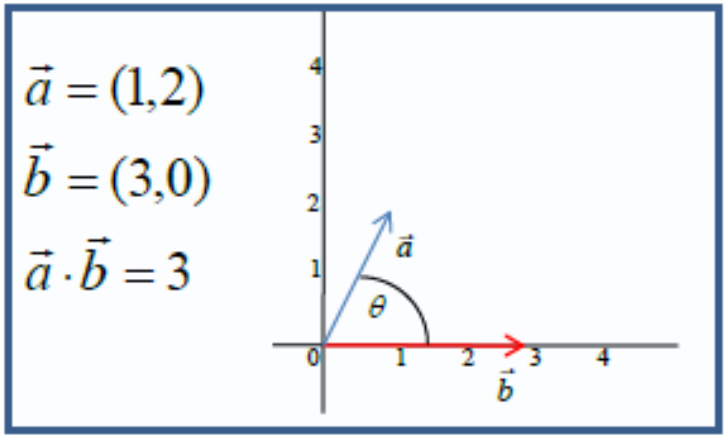
Consider vectors $\vec{a} = (a_1, a_2, \ldots, a_n)$ and $\vec{b} = (b_1, b_2, \ldots, b_n)$

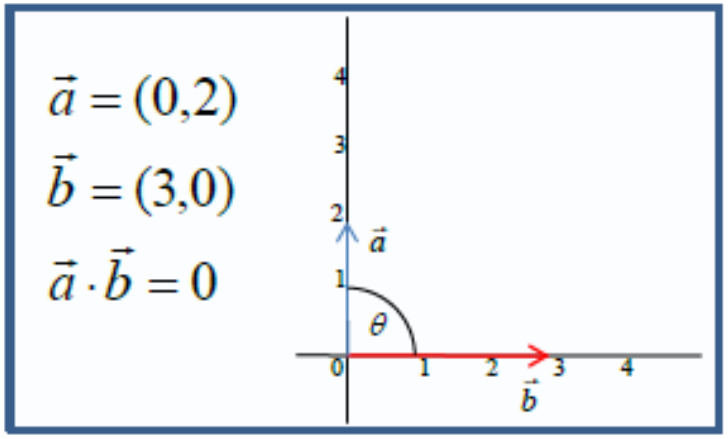
Define dot product: $\langle \vec{a}, \vec{b} \rangle = \vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2 + \ldots + a_n b_n = \sum_{i=1}^{n} a_i b_i$

The law of cosines says that $\vec{a} \cdot \vec{b} = \| \vec{a} \|_2 \| \vec{b} \|_2 \cos\theta$

where θ is the angle between $\vec{a}$ and $\vec{b}$.

Therefore, when the vectors are perpendicular $\vec{a} \cdot \vec{b} = 0$.





Basic operation on vectors

Dot product(continued)

$$\langle \vec{a}, \vec{b} \rangle = \vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2 + \ldots + a_n b_n = \sum_{i=1}^{n} a_i b_i$$

Property: $\vec{a} \cdot \vec{a} = a_1 a_1 + a_2 a_2 + \ldots + a_n a_n = \left\| \vec{a} \right\|_2^2$

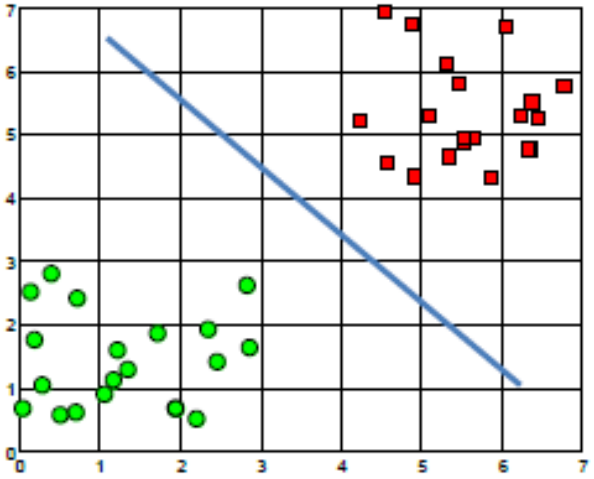
- In the classical regression equation $y = \vec{w} \cdot \vec{x} + b$ the response variable y is just a dot product of the vector representing patient characteristics ($\vec{x}$) and the regression weights vector ($\vec{w}$) which is common across all patients plus an offset *b*.

Hyperplanes as decision surfaces

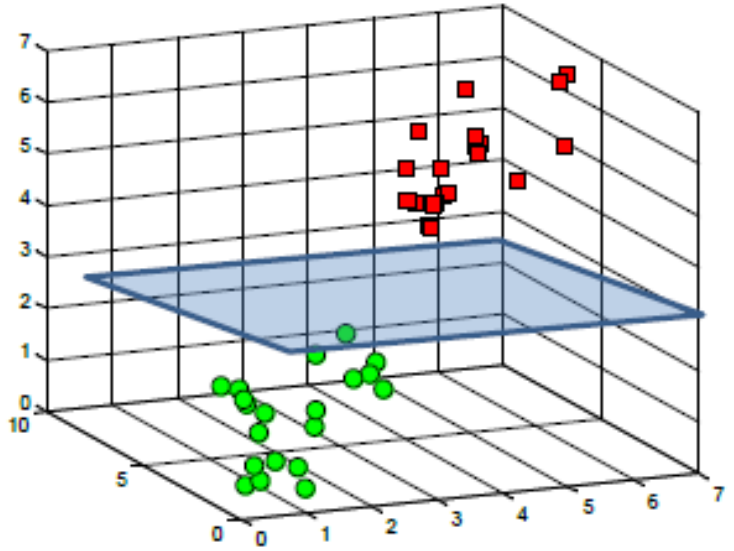
A hyperplane is a linear decision surface that splits the space into two parts;

It is obvious that a hyperplane can be used a binary

A hyperplane in $\mathbb{R}^2$ is a line



A hyperplane in $\mathbb{R}^3$ is a plane

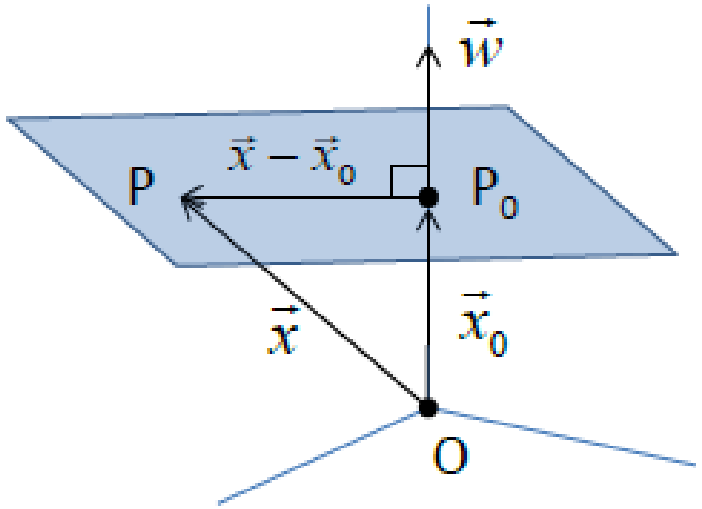


A hyperplane in $\mathbb{R}^n$ is an n-1 dimensional subspace

Equation of a hyperplane

An equation of a hyperplane is defined by a point (P0) and a perpendicular vector to the plane ($\vec{w}$) at that point.

Consider the case of $\mathbb{R}^3$:



Define vectors $\vec{x}_0 = \overrightarrow{OP_0}$ and $\vec{x} = \overrightarrow{OP}$, where *P* is an arbitrary point on a hyperplane

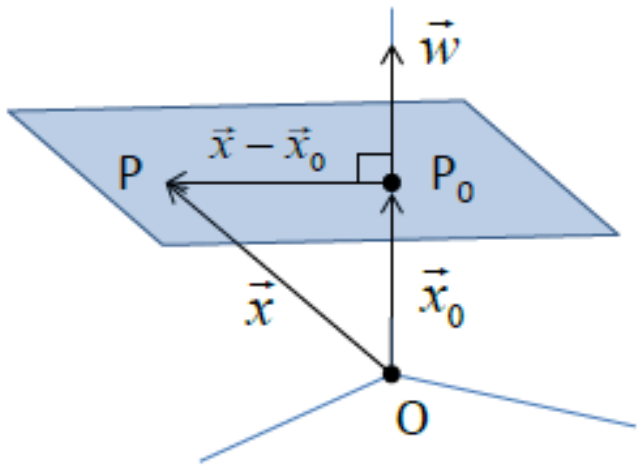
Equation of a hyperplane

A condition for *P* to be on the plane is that the vector $\vec{x} - \overrightarrow{x_0}$ is perpendicular to $\vec{w}$:

$$\vec{w}\cdot(\vec{x}-\vec{x}_0)=0 \quad \text{or}$$

$$\vec{w}\cdot\vec{x}-\vec{w}\cdot\vec{x}_0=0 \quad \text{define } b=-\vec{w}\cdot\vec{x}_0$$

$$\boxed{\vec{w}\cdot\vec{x}+b=0}$$



Equation of a hyperplane

Example:

$\vec{w} = (4,-1,6)$

$P_0 = (0,1,-7)$

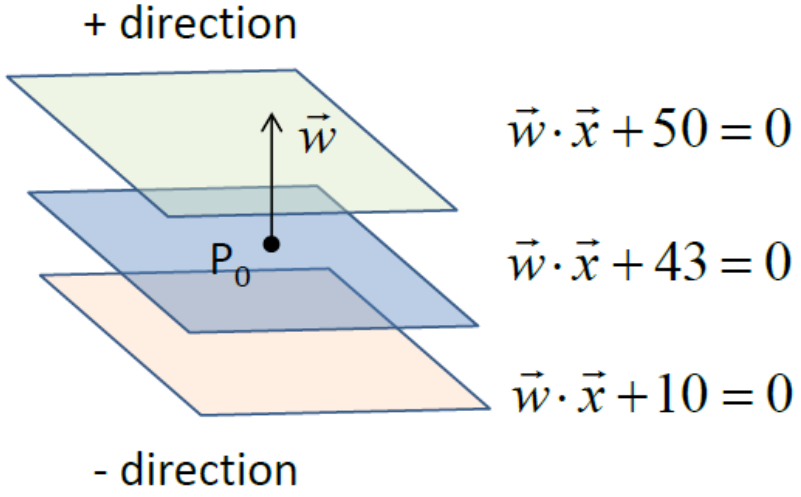
$b = -\vec{w} \cdot \vec{x}_0 = -(0-1-42) = 43$

$\Rightarrow \vec{w} \cdot \vec{x} + 43 = 0$

$\Rightarrow (4,-1,6) \cdot \vec{x} + 43 = 0$

$\Rightarrow (4,-1,6) \cdot (x_{(1)}, x_{(2)}, x_{(3)}) + 43 = 0$

$\Rightarrow 4x_{(1)} - x_{(2)} + 6x_{(3)} + 43 = 0$

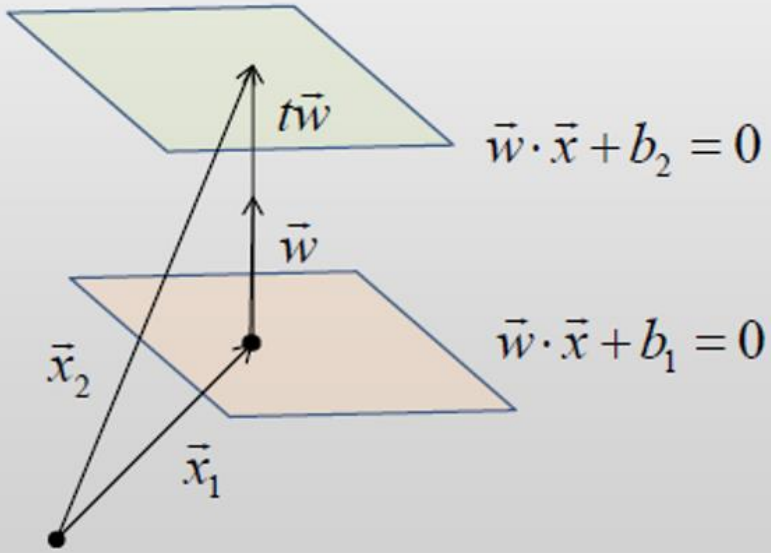


What happens if the *b* coefficient changes?

The hyperplane moves along the direction of $\vec{w}$. We obtain "parallel hyperplanes". $\vec{w} \cdot \vec{x} + b$

The distance between two parallel hyperplanes $\vec{w} \cdot \vec{x} + b_1$ and $\vec{w} \cdot \vec{x} + b_2$ is equal to $D = \frac{|b_1 - b_2|}{\|\vec{w}\|}$

Distance between two parallel hyperplanes



$$\vec{x}_2 = \vec{x}_1 + t\vec{w}$$

$$D = \left\|t\vec{w}\right\| = \left|t\right|\left\|\vec{w}\right\|$$

$$\vec{w}\cdot\vec{x}_2 + b_2 = 0$$

$$\vec{w}\cdot(\vec{x}_1 + t\vec{w}) + b_2 = 0$$

$$\vec{w}\cdot\vec{x}_1 + t\left\|\vec{w}\right\|^2 + b_2 = 0$$

$$(\vec{w}\cdot\vec{x}_1 + b_1) - b_1 + t\left\|\vec{w}\right\|^2 + b_2 = 0$$

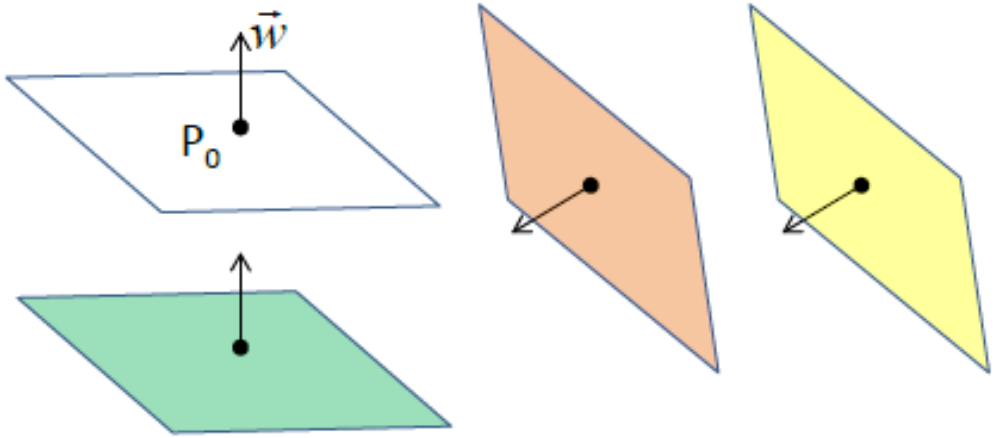
$$-b_1 + t\left\|\vec{w}\right\|^2 + b_2 = 0$$

$$t = (b_1 - b_2)/\left\|\vec{w}\right\|^2$$

$$\Rightarrow D = \left|t\right|\left\|\vec{w}\right\| = \left|b_1 - b_2\right|/\left\|\vec{w}\right\|$$

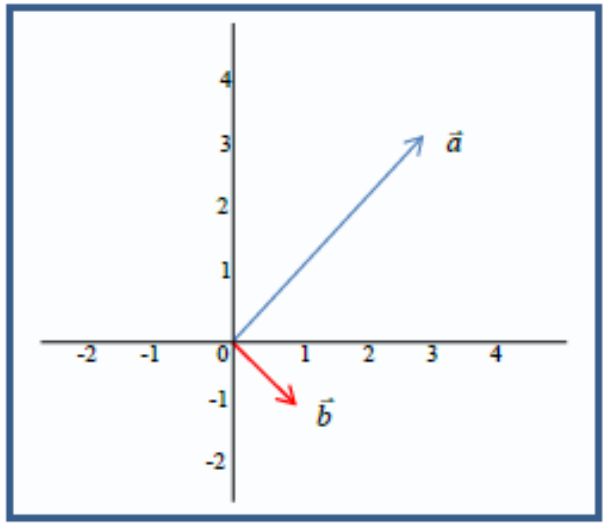
QUIZ

1) Consider a hyperplane shown with white. It is defined by equation: $\vec{w} \cdot \vec{x} + 10 = 0$
Which of the three other hyperplanes can be defined by equation: $\vec{w} \cdot \vec{x} + 3 = 0$?



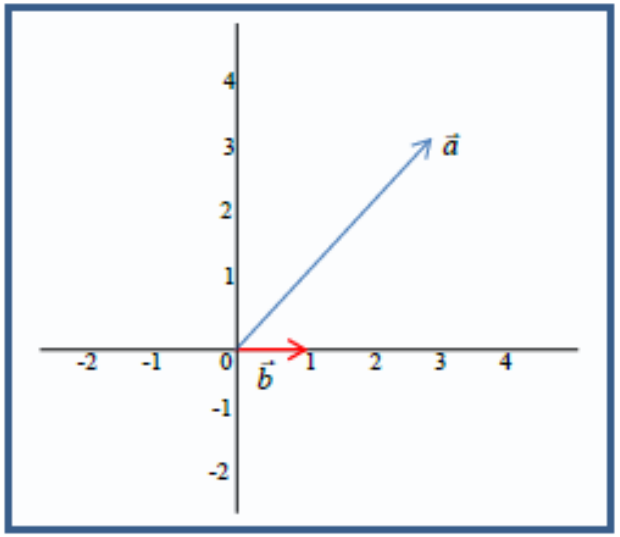
- Orange
- Green
- Yellow

2) What is the dot product between vectors $\vec{a} = (3,3)$ and $\vec{b} = (1,-1)$?

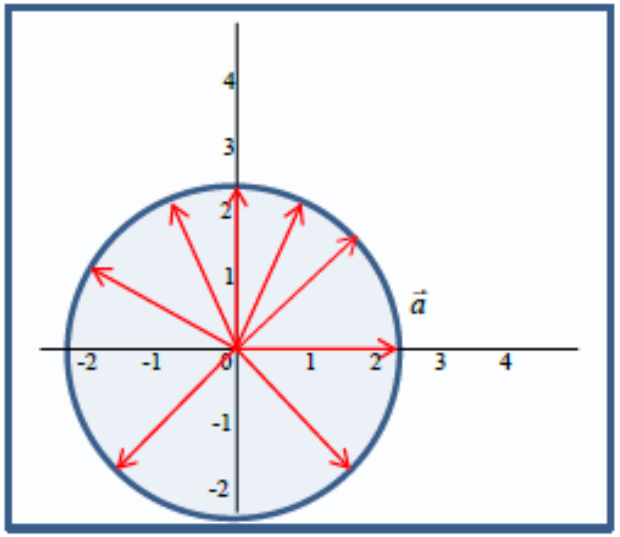


QUIZ

3) What is the dot product between vectors $\vec{a} = (3,3)$ and $\vec{b} = (1,0)$?



4) What is the length of a vector $\vec{a} = (2,0)$ and what is the length of all other red vectors in the figure?



What is a Hyperplane?

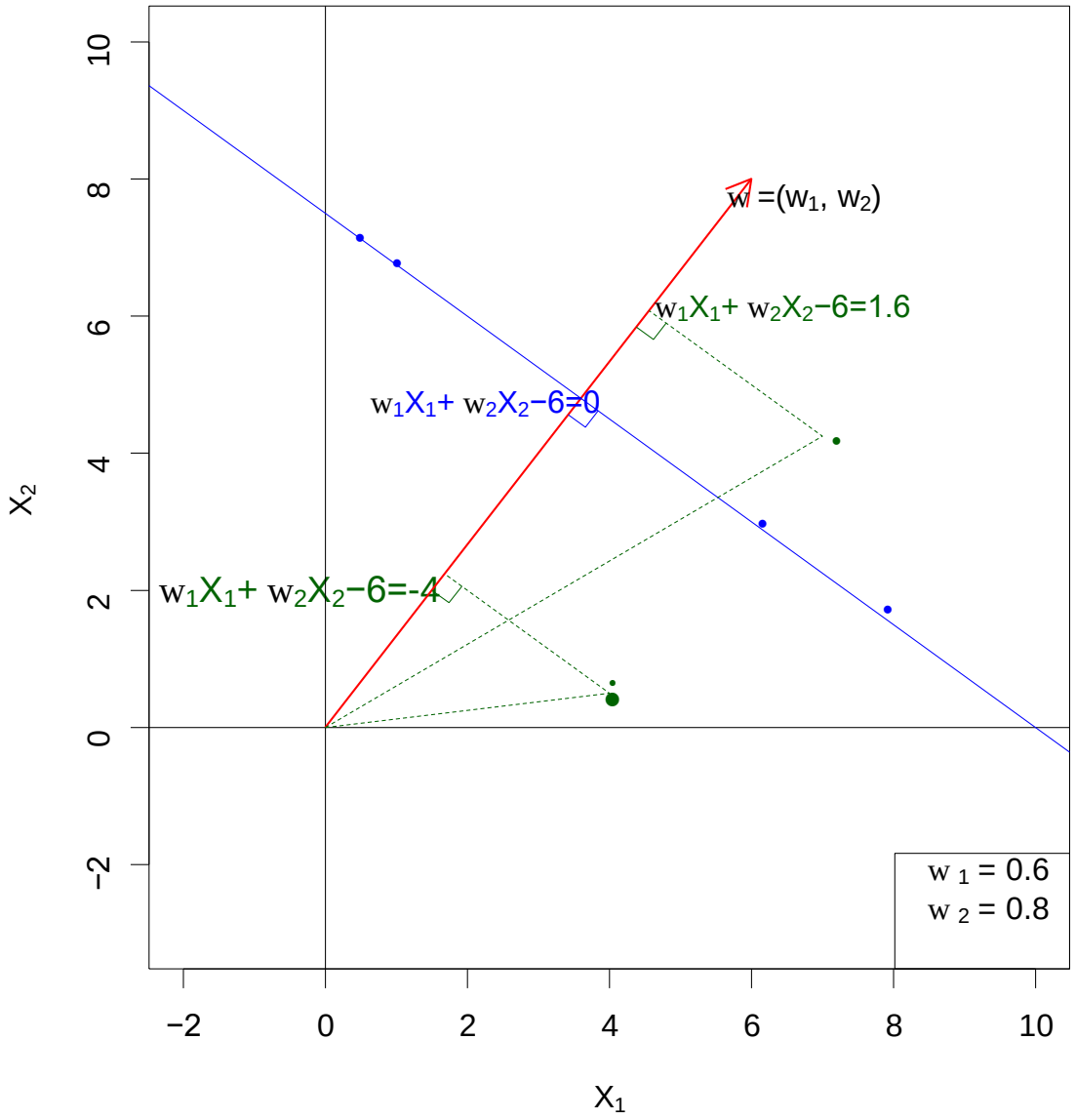
- A hyperplane in p dimensions is a flat affine subspace of dimension $p - 1$.
- In general the equation for a hyperplane has the form:

$$b + w_1 X_1 + w_2 X_2 + \ldots + w_p X_p = 0$$

- In $p = 2$ dimensions a hyperplane is a line.
- If $b = 0$, the hyperplane goes through the origin, otherwise not.
- The vector $\vec{w} = (w1, w2, \cdots, wp)$ is called the **normal vector**

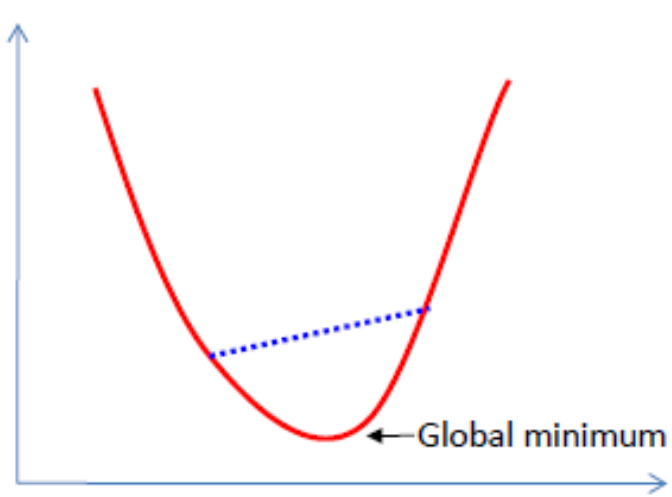
— it points in a direction orthogonal to the surface of a hyperplane.

Hyperplane in 2 Dimensions

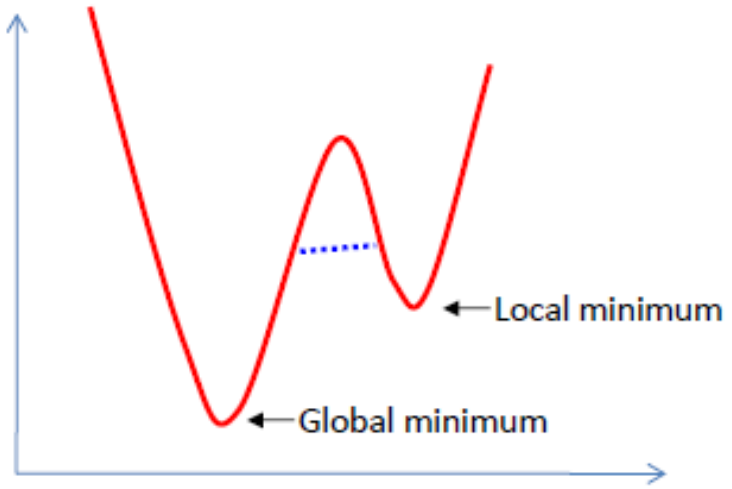


Convex functions

- A function is called *convex* if the function lies below the straight line segment connecting two points, for any two points in the interval.
- Property: Any local minimum is a global minimum!



Convex function



Non-convex function

Quadratic programming (QP)

- *Quadratic programming* (QP) is a special optimization problem: the function to optimize ("*objective*") is quadratic, subject to linear *constraints*.
- *Convex QP problems* have convex objective functions.
- These problems can be solved easily and efficiently by greedy algorithms (because every local minimum is a global minimum).

Example QP problem

Consider $\vec{x} = (x_1, x_2)$

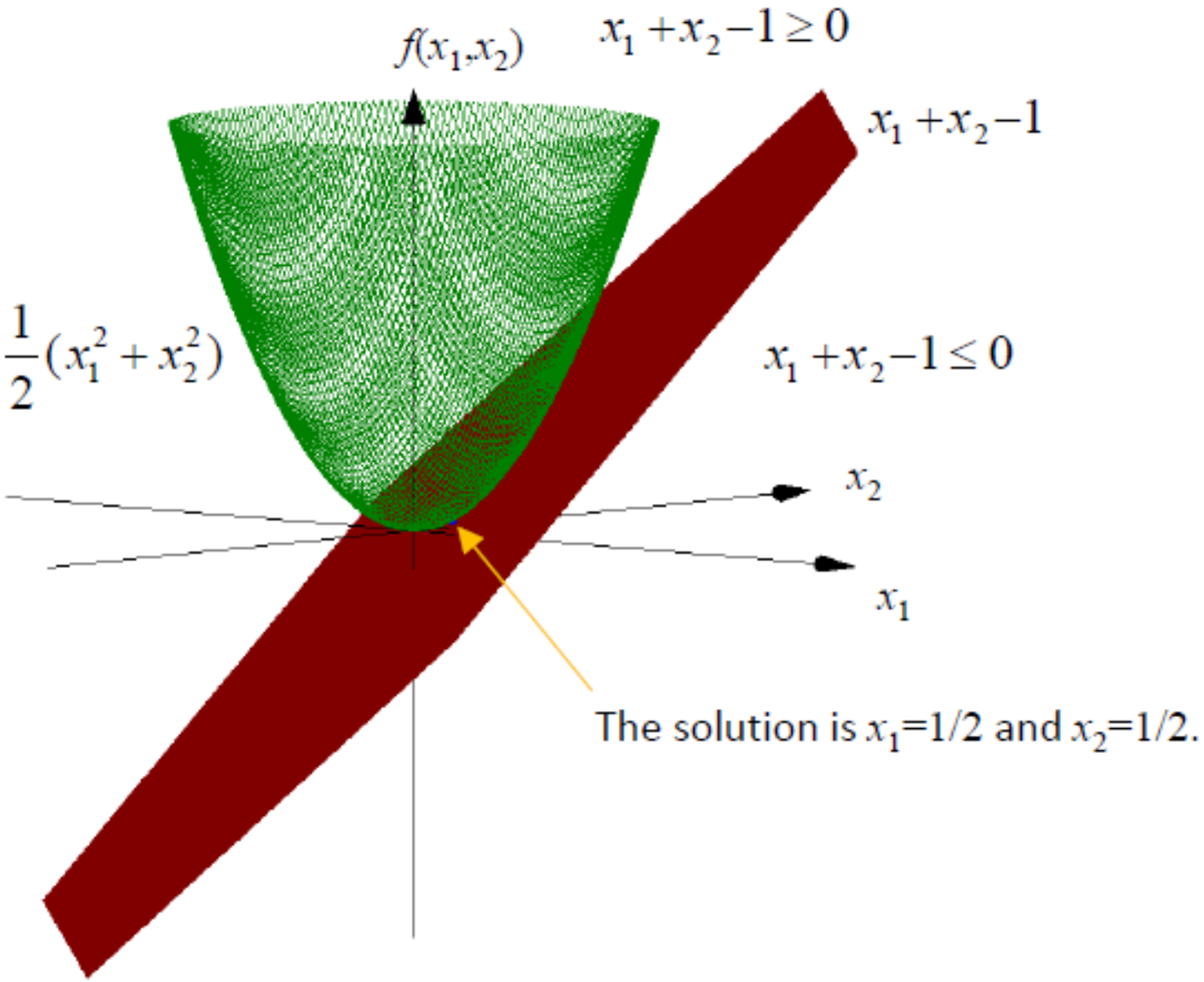
Minimize $\underbrace{\frac{1}{2} \| \vec{x} \|_2^2}_{\text{quadratic objective}}$ subject to $\underbrace{x_1 + x_2 - 1 \geq 0}_{\text{linear constraints}}$

This is QP problem, and it is a convex QP

We can rewrite it as:

Minimize $\underbrace{\frac{1}{2}(x_1^2 + x_2^2)}_{\text{quadratic objective}}$ subject to $\underbrace{x_1 + x_2 - 1 \geq 0}_{\text{linear constraints}}$

Example QP problem



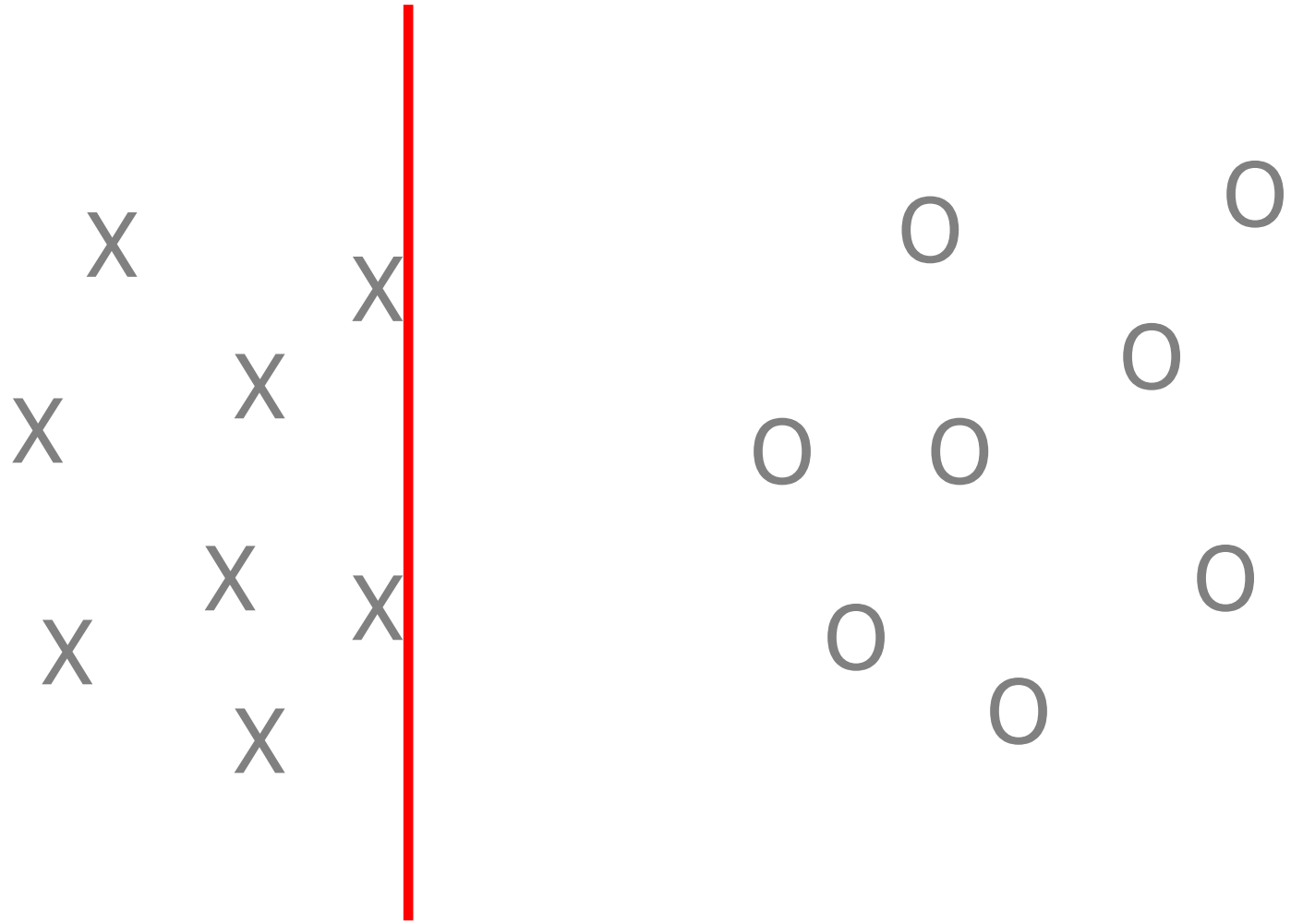
Separable Hyperplanes

Imagine a situation where you have a two classes classification problem with two predictors X_1 and X_2.

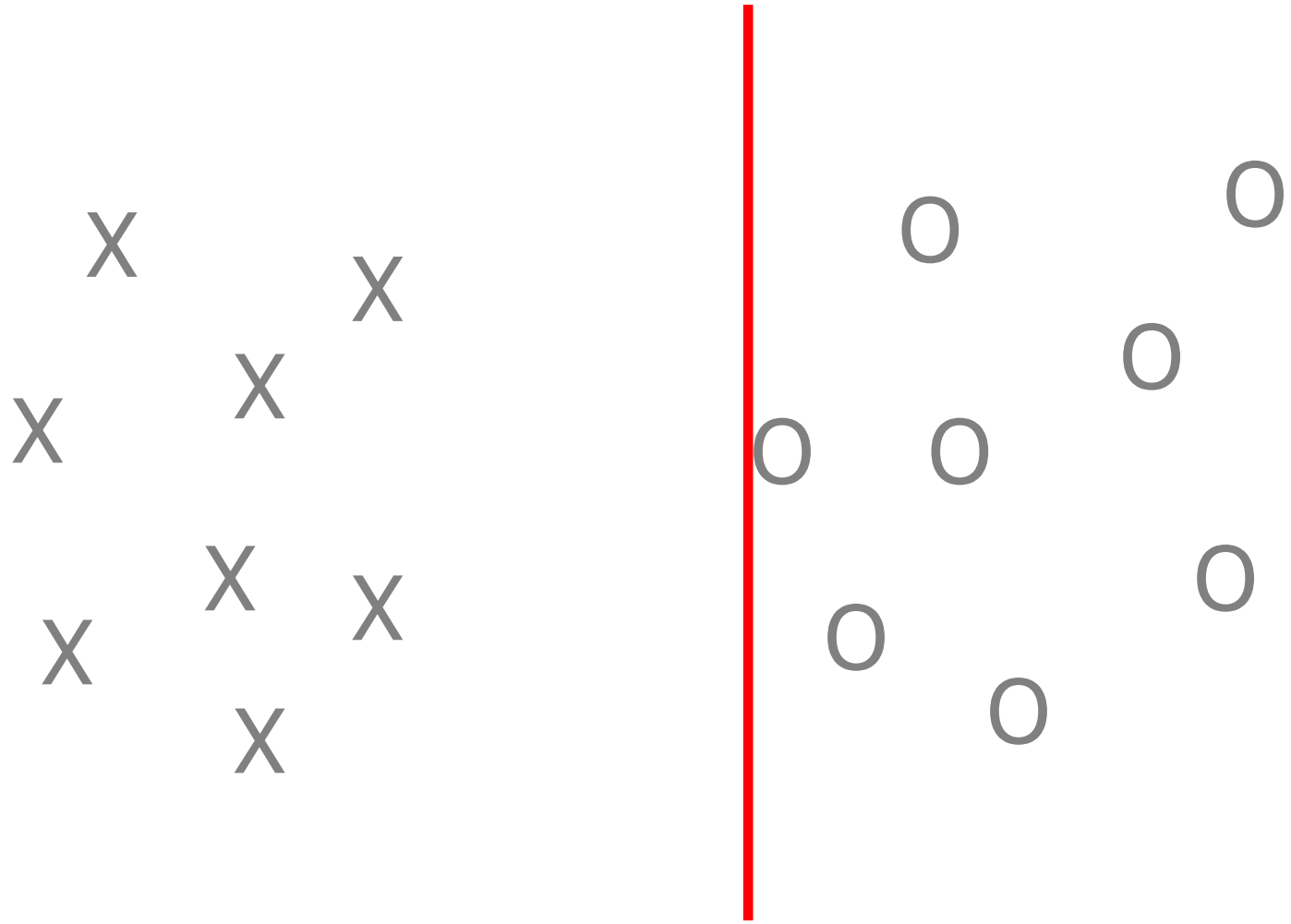
Suppose that the two classes are "linearly separable" i.e. one can draw a straight line in which all points on one side belong to the first class and points on the other side to the second class.

Intuitions

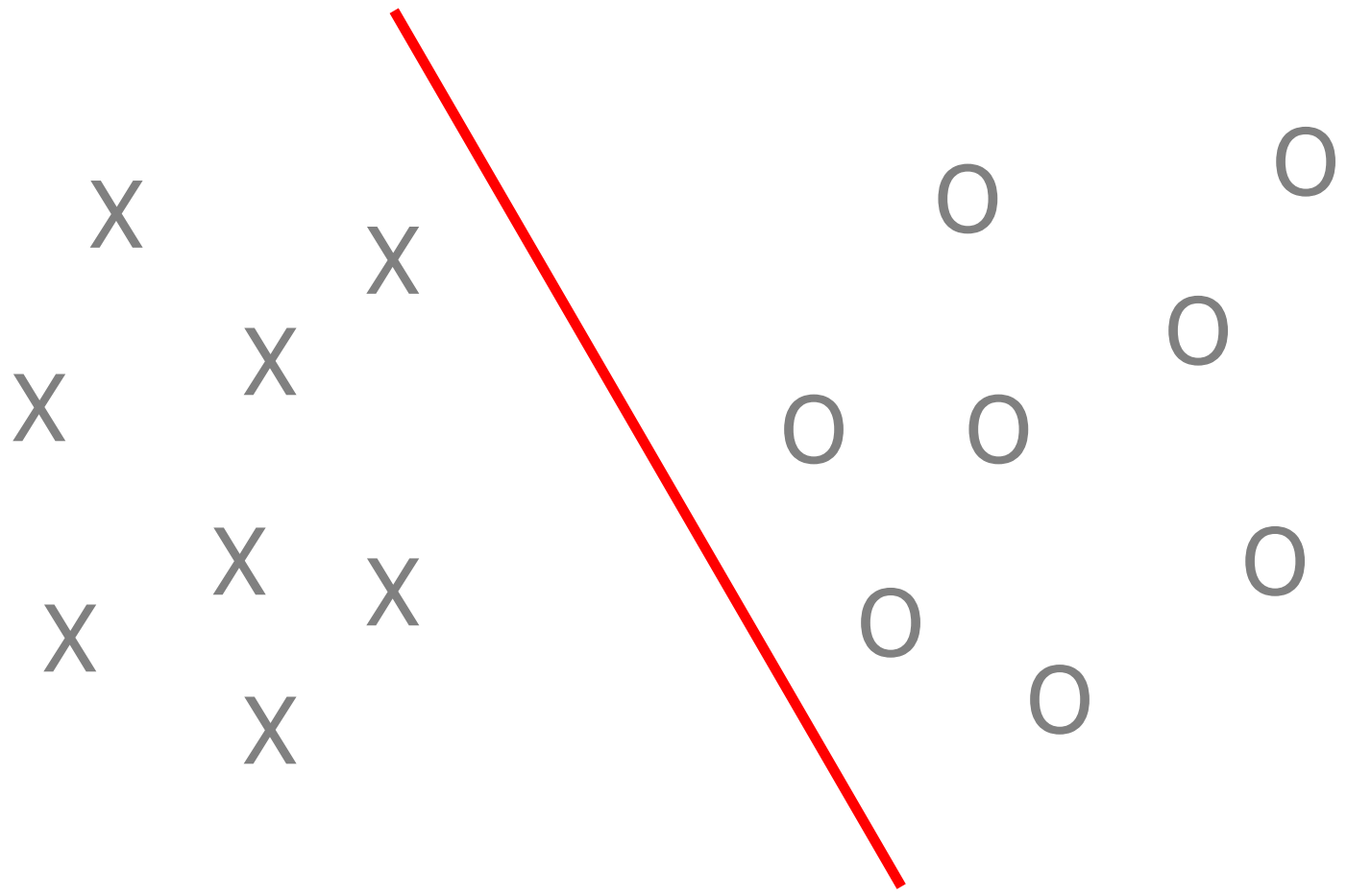
Intuitions



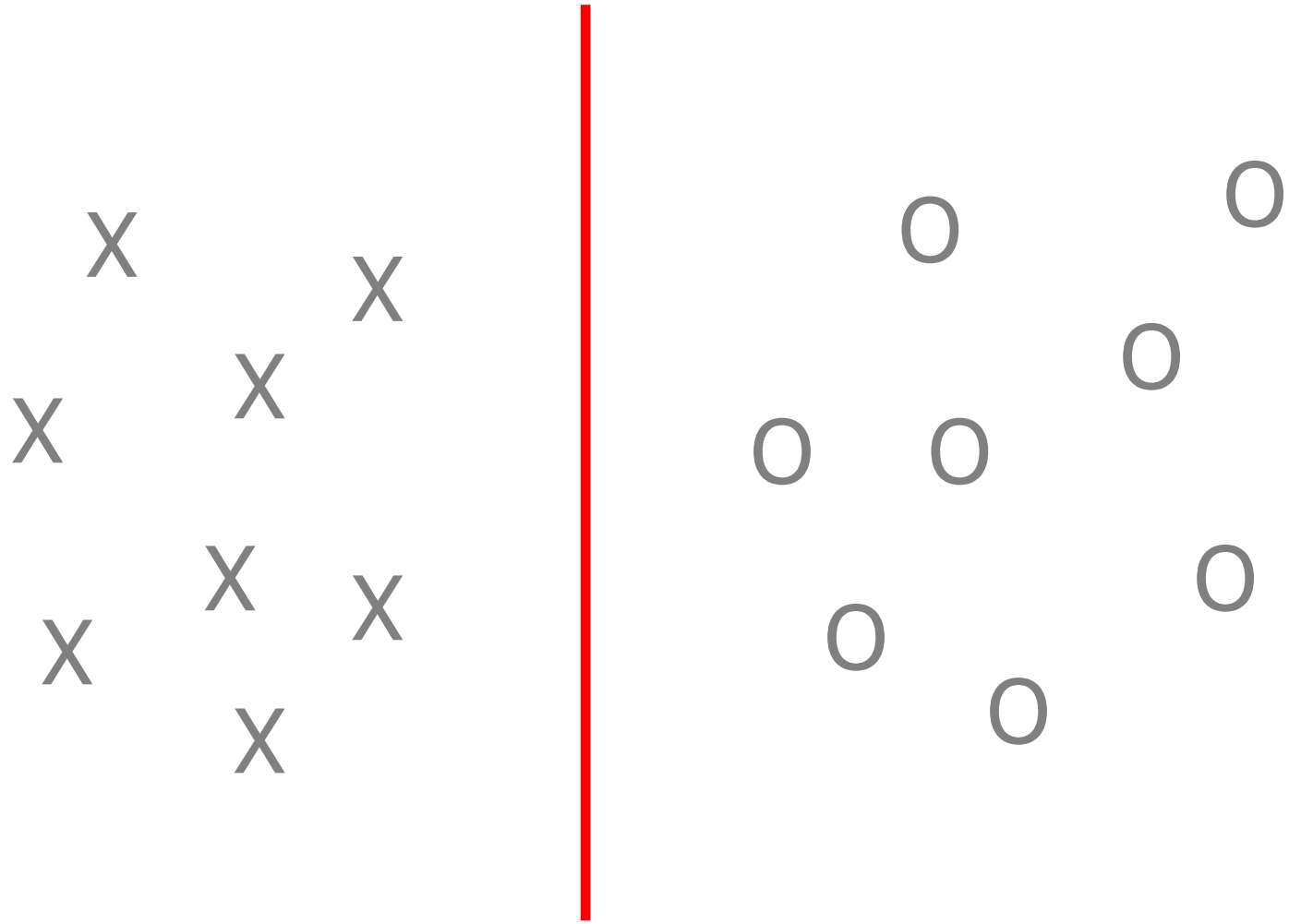
Intuitions



Intuitions



A “Good” Separator



Separable Hyperplanes

Then a natural approach is to find the straight line that gives the biggest separation between the classes i.e. the points are as far from the line as possible

This is the basic idea of a support vector classifier.

SVM

Basic idea of support vector machines:

- Optimal hyperplane for linearly separable patterns
- Extend to patterns that are not linearly separable by transformations of original data to map into new space – the Kernel function

SVM

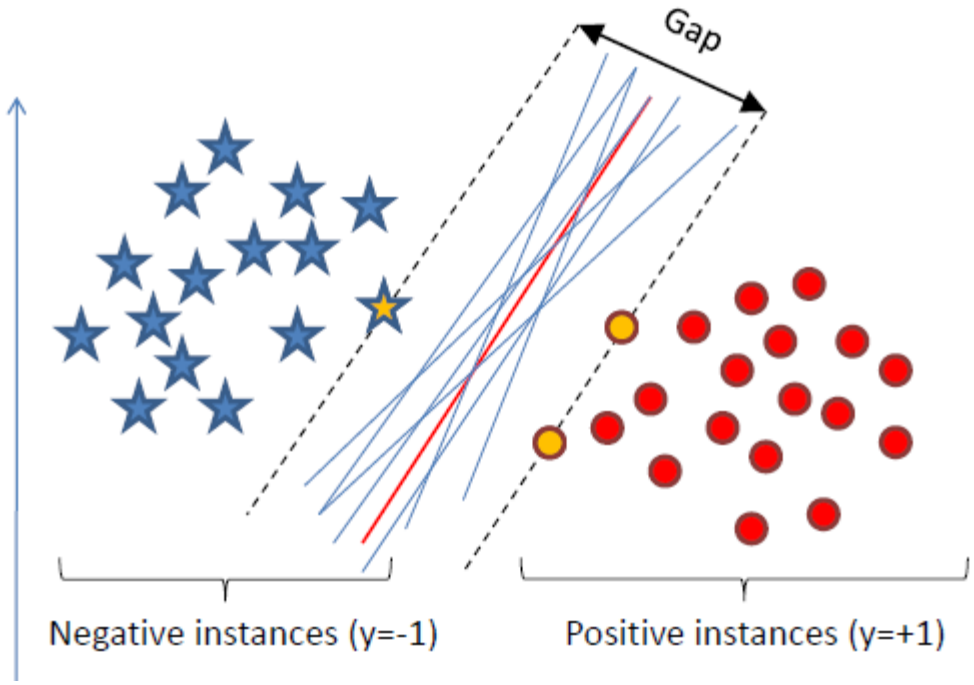
Here we approach the two-class classification problem in a direct way:

We try and find a plane that separates the classes in the feature space.

If we cannot, we get creative in two ways:

- We soften what we mean by "separates", and
- We enrich and enlarge the feature space so that separation is possible.

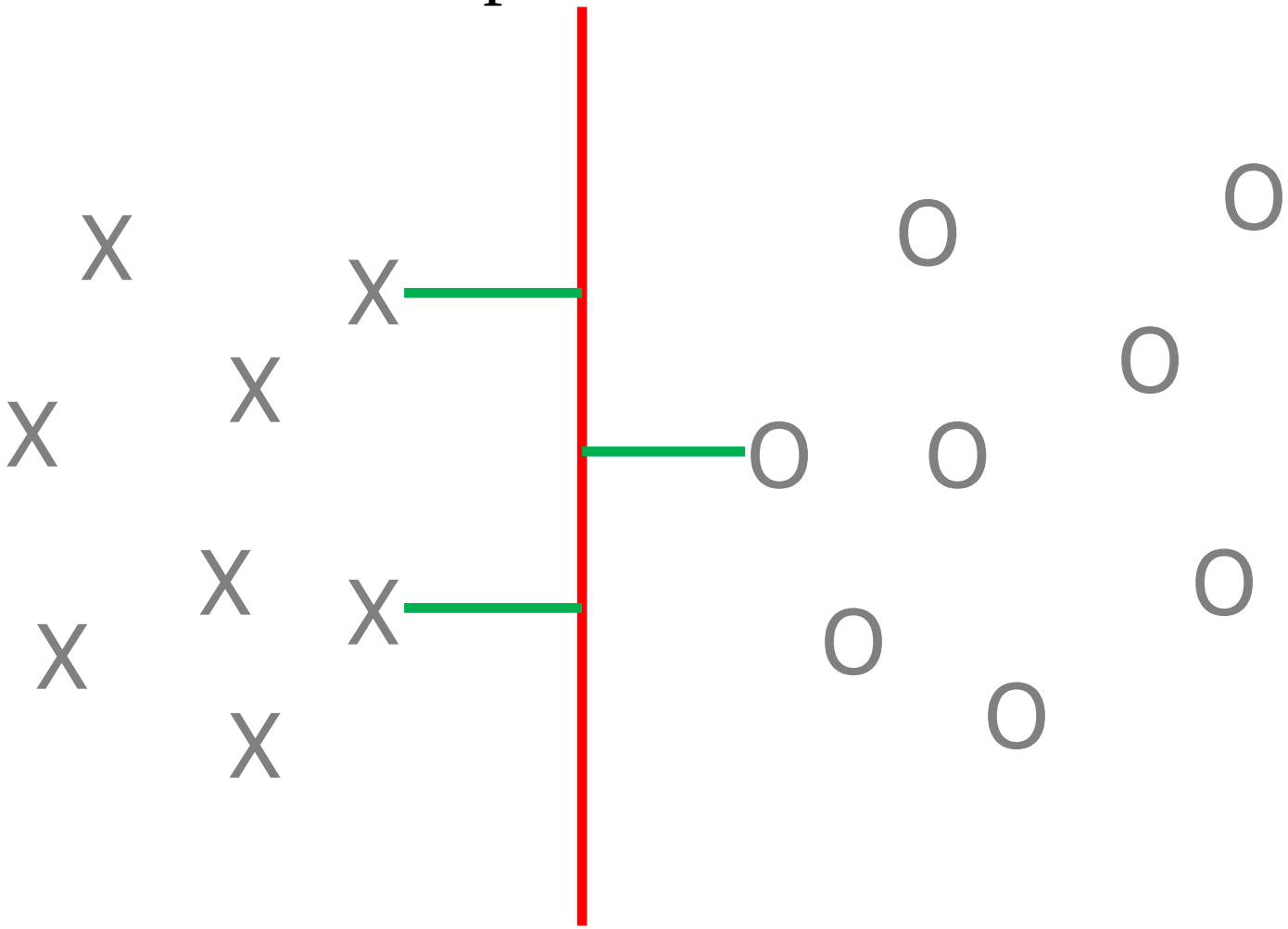
Separating Hyperplanes



- If $f(X) = b + w_1 X_1 + \cdots + w_p X_p$, then $f(X) > 0$ for points on one side of the hyperplane, and $f(X) < 0$ for points on the other.
- If we code the colored points as $Y_i = +1$ for red, say, and $Y_i = -1$ for blue, then if $Y_i \cdot f(X_i) > 0$ for all i, $f(X) = 0$ defines a separating hyperplane.

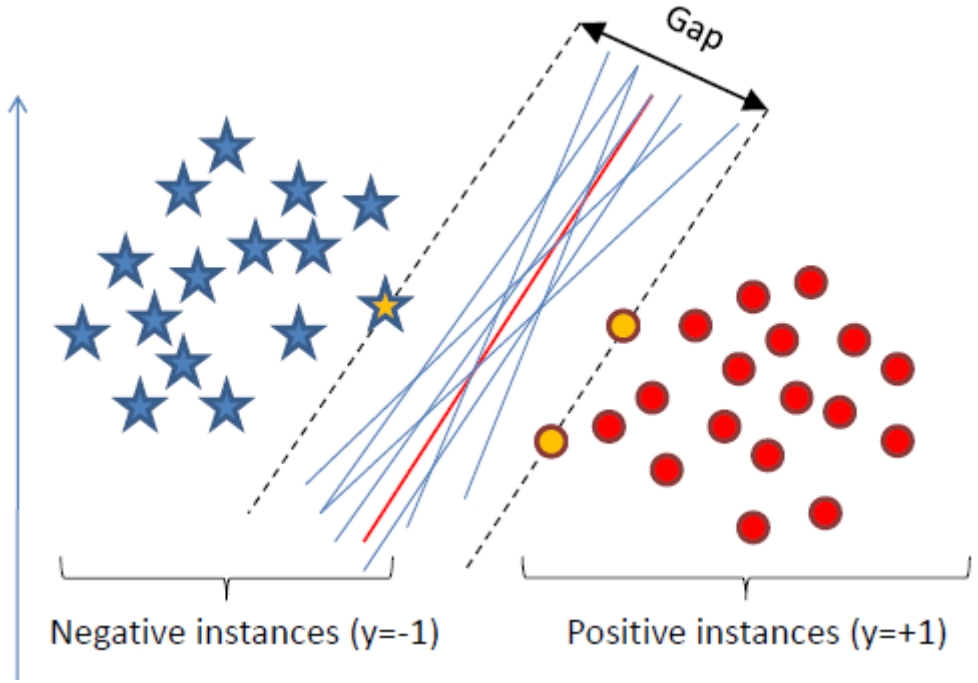
Maximizing the Margin

Margin: minimal distance of the Hyperplane to the nearest data point on each side



SVM

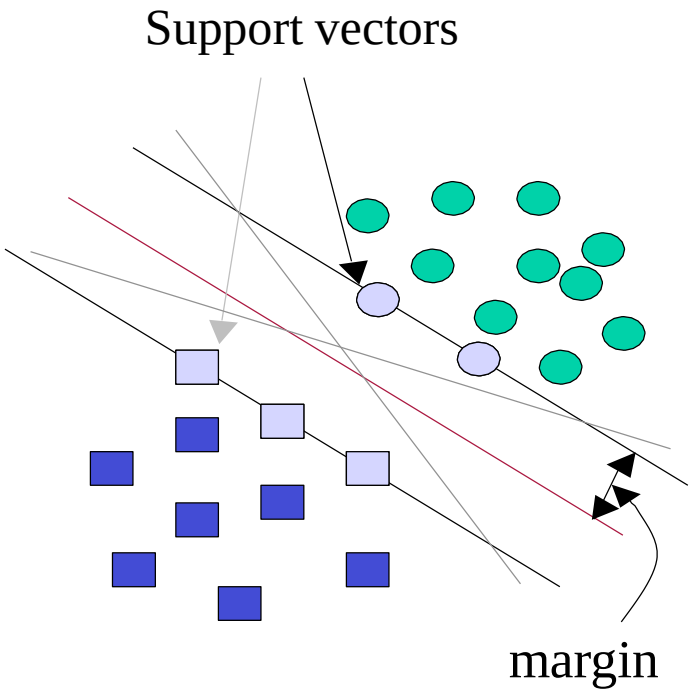
Among all separating hyperplanes, find the one that makes the biggest gap or margin between the two classes.



If the points on the boundaries are not informative (e.g., due to noise), SVMs will not do well.

SVM

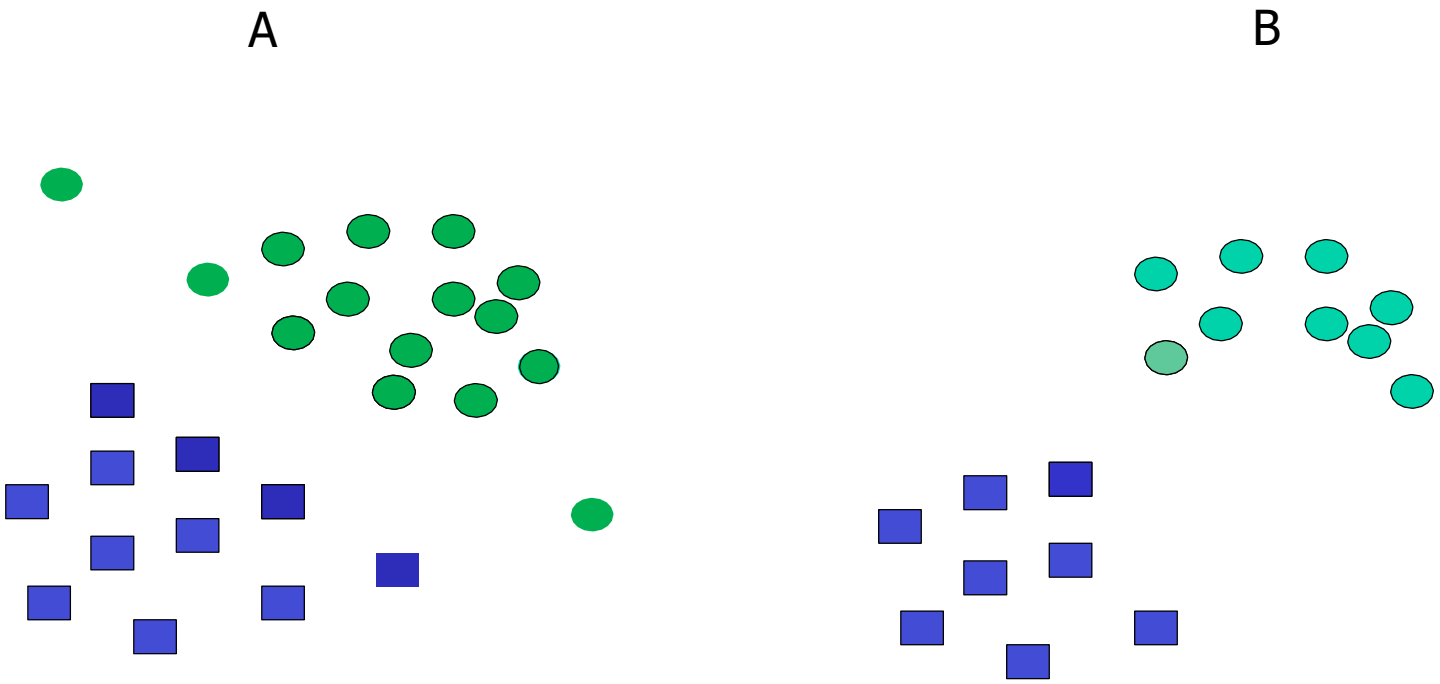
- SVMs maximize the margin around the separating hyperplane.
- The decision function is fully specified by a (usually very small) subset of training samples, the **support vectors.**
- The maximal-margin hyperplane depends only by the support vectors and not by the other observations.



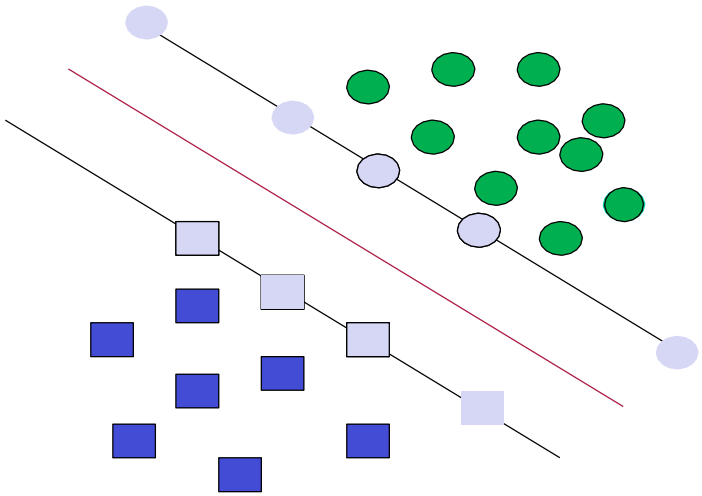
Support Vectors

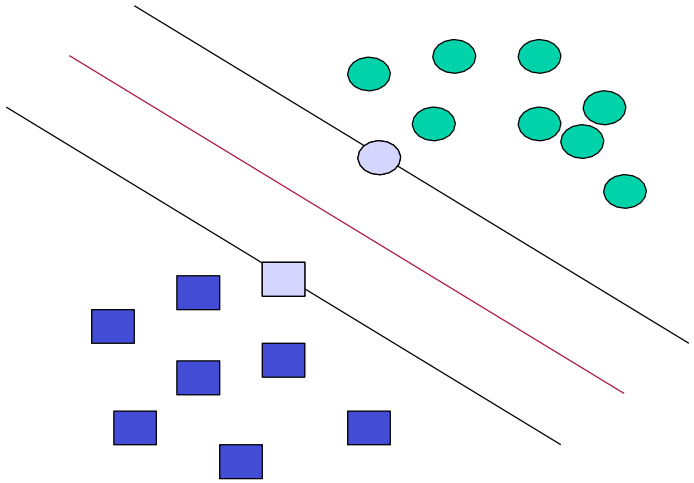
- Support vectors are the data points that lie closest to the decision surface (or hyperplane).
- They are the data points **most difficult to classify**.
- They have direct bearing on the optimum location of the decision surface.
- Support vectors are the elements of the training set that would **change the position of the dividing hyperplane** if removed.
- This problem can be formulated as a **Quadratic programming problem** which can be **easily solved** by standard methods

Support Vectors?



Support Vectors





SVM formalization

- Assume that the functional margin of each data item is at least 1, then the following two constraints follow for a training set $\{(x_i, y_i)\}$
- The gap is distance between parallel hyperplanes:

$\mathbf{w}^T\mathbf{x}_i + (b+1) = 0$

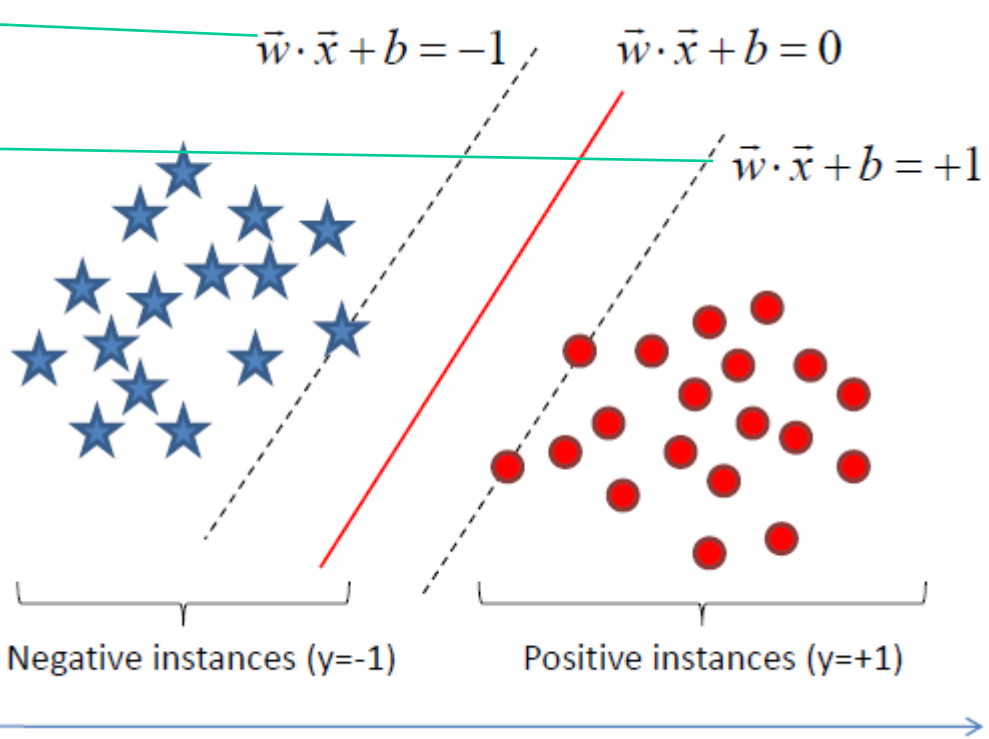
$\mathbf{w}^T\mathbf{x}_j + (b-1) = 0$

- We know that

$$D = |b_1 - b_2| / \|\vec{w}\|$$

Therefore:

$$D = 2 / \|\vec{w}\|$$



SVM formalization

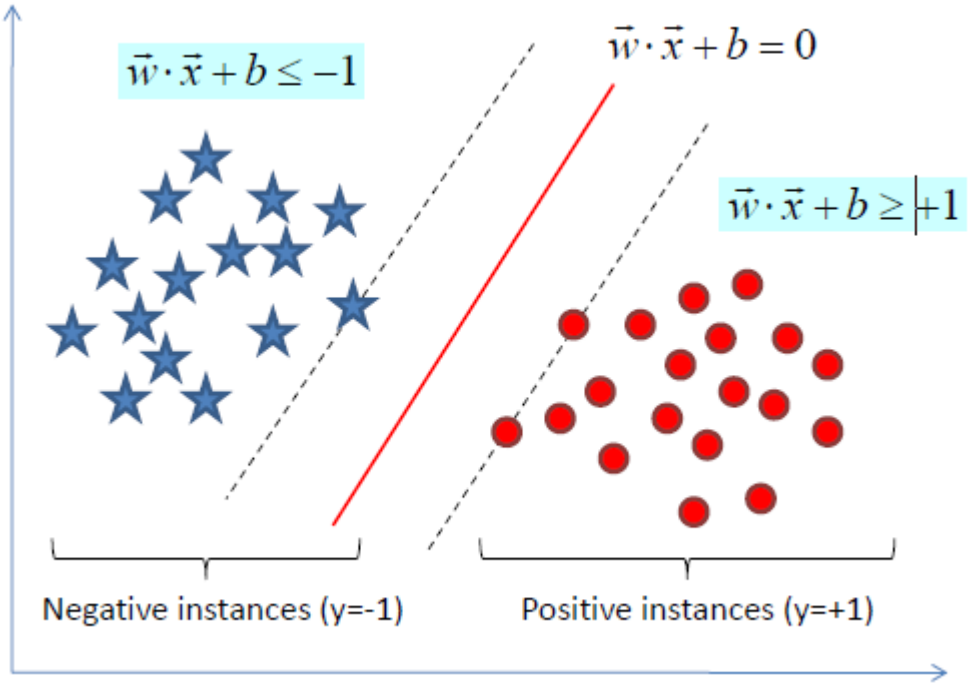
- In addition we need to impose constraints so that all instances are correctly classified

$\mathbf{w}^{\mathrm{T}}\mathbf{x_i} + b \geq 1 \quad \text{if } y_i = 1$

$\mathbf{w}^{\mathrm{T}}\mathbf{x_i} + b \leq -1 \quad \text{if } y_i = -1$

Or equivalently:

$y_i(\mathbf{w}^{\mathrm{T}}\mathbf{x_i} + b) \geq 1$



- Since we want to maximize the gap, we need to minimize $\| \boldsymbol{w} \|$
  or equivalently minimize $\frac{1}{2}\| \boldsymbol{w} \|^2$

SVM formalization

- Constrained optimization problem

$$\underset{b,\,\mathbf{w}}{\text{minimize}} \; \frac{1}{2}\| \boldsymbol{w} \|^2 \quad \text{subject to}$$

$$y_i (b + \mathbf{w}^T \mathbf{x}_i) - 1 \geq 0 \quad \text{for all} \quad i = 1, \ldots, N.$$

- This is called "**primal formulation of linear SVMs**".

- It is a convex optimization problem (quadratic criterion, linear inequality constraint), that can be solved efficiently.

SVM formalization

- For support vectors, the inequality becomes an equality
- Then, since each example's distance from the hyperplane is

$$r = y\frac{\boldsymbol{w}^T\boldsymbol{x}+b}{\|\boldsymbol{w}\|}$$

- The margin is:

$$\rho = \frac{1}{\|\boldsymbol{w}\|}$$

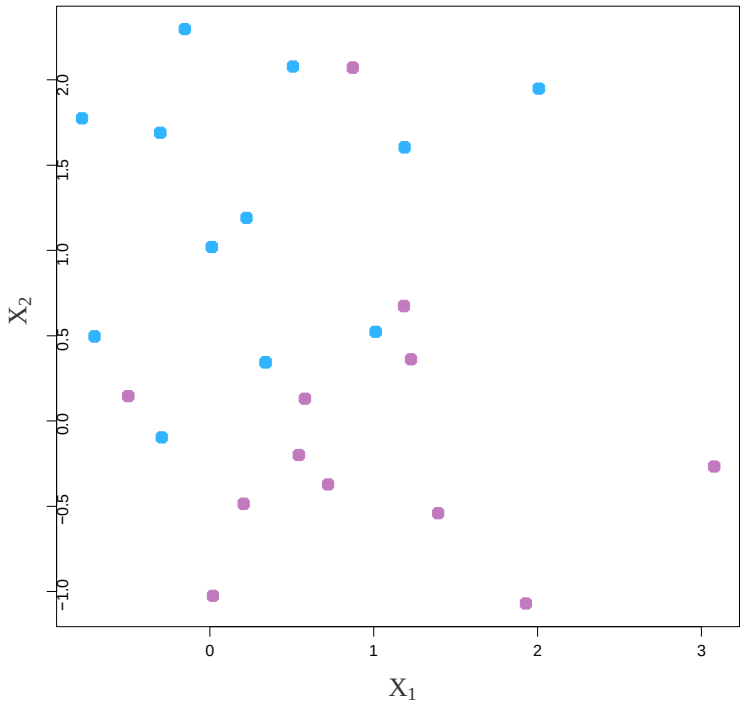
More Than Two Attributes

- This idea works just as well with **more than two attributes**.
- For example, with three attributes you want to find the **plane** that produces the largest separation between the classes.
- With more than three dimensions it becomes hard to visualize a plane but it still exists. In general they are called **hyper-planes**.

Non-Separating Classes

- Of course in practice it is **not** usually possible to find a hyper-plane that **perfectly separates** two classes.
- In other words, for any straight line or plane that I draw there will always be at least some points on the wrong side of the line.
- In this situation we try to find the plane that gives the **best separation** between the points that are correctly classified subject to the points on the wrong side of the line not being off by too much.
- It is easier to see with a picture!

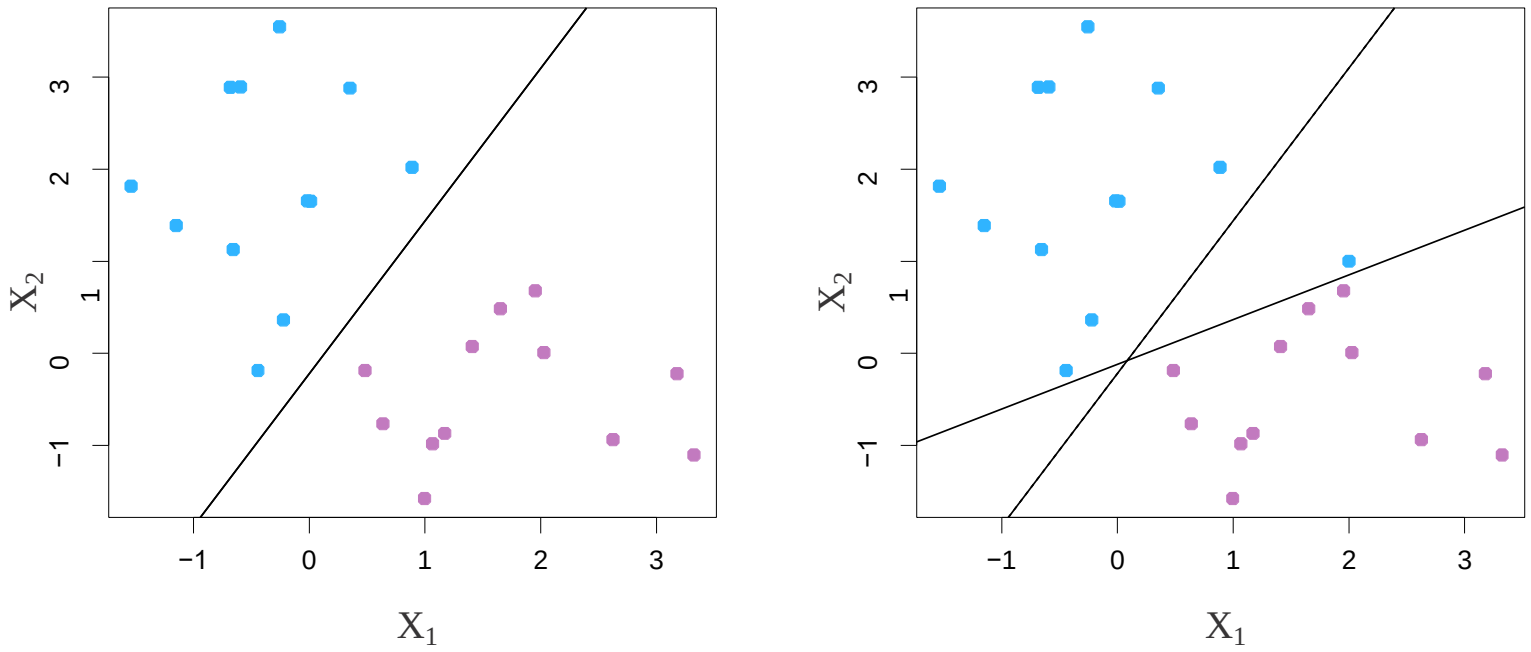
Non-separable Data



The data on the left are **not separable** by a linear boundary.

This is often the case, unless $N < p$
where N is the number of elements of the dataset and p is the number of features of these elements.

Noisy Data



Sometimes the data are **separable**, but **noisy**. This can lead to a poor solution for the maximal-margin classifier.

Support Vector Classifier

Soft margin

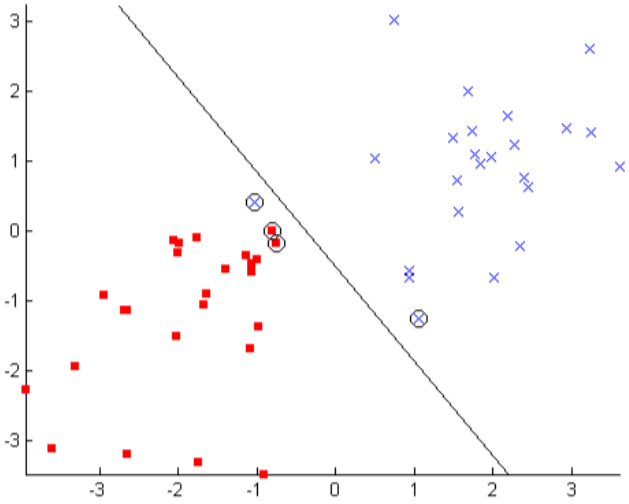
$$\begin{aligned} \min_{w,b,\varepsilon} \quad & \frac{1}{2} w^T w + C \sum_{i=1}^{n} \epsilon_i \\ \text{subject to} \quad & y_i \left(w^T x_i + b \right) \geq 1 - \epsilon_i, \quad \forall i \\ & \epsilon_i \geq 0, \quad \forall i \end{aligned}$$

In the following figures, C= 0.1, 1, 10, or 100. The SVM decision boundary has been drawn and all support vectors are circled.

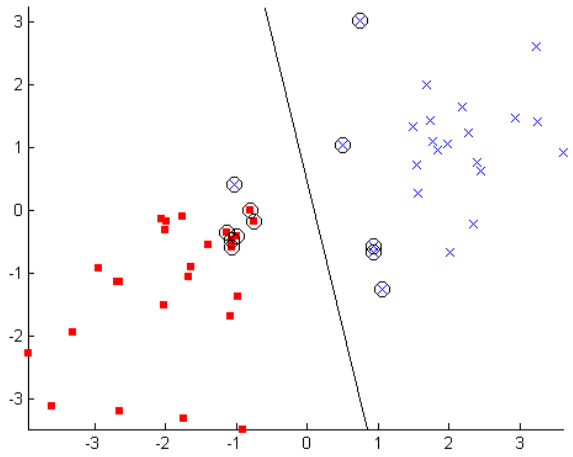
The support vector classifier maximizes a soft margin.

Support Vector Classifier

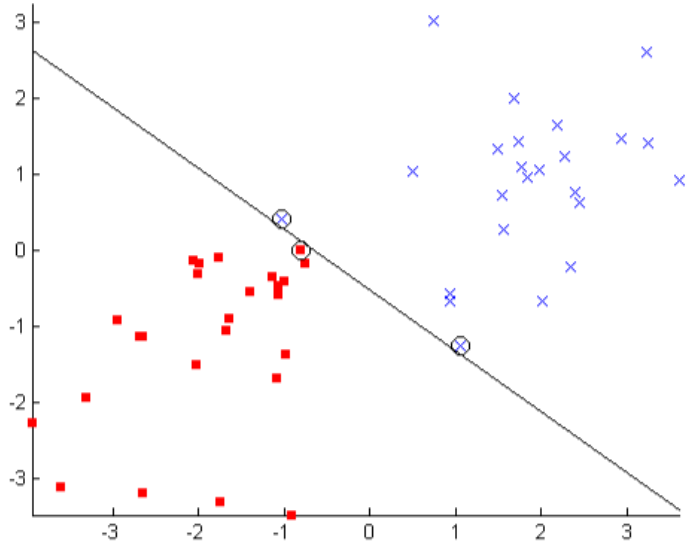
a



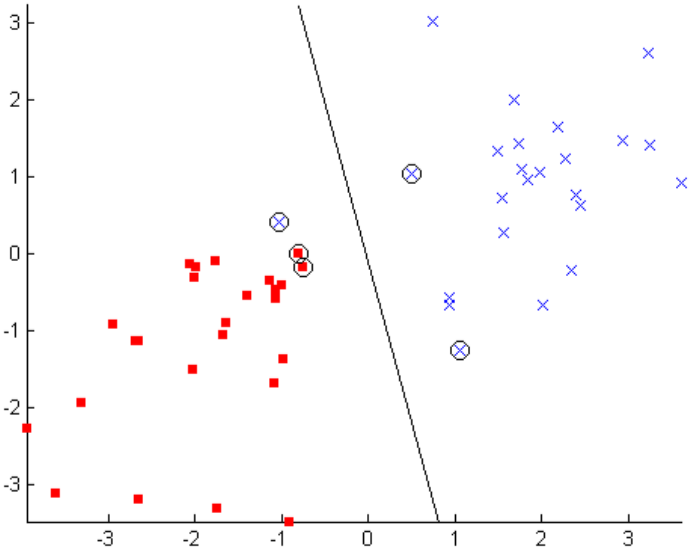
b



c

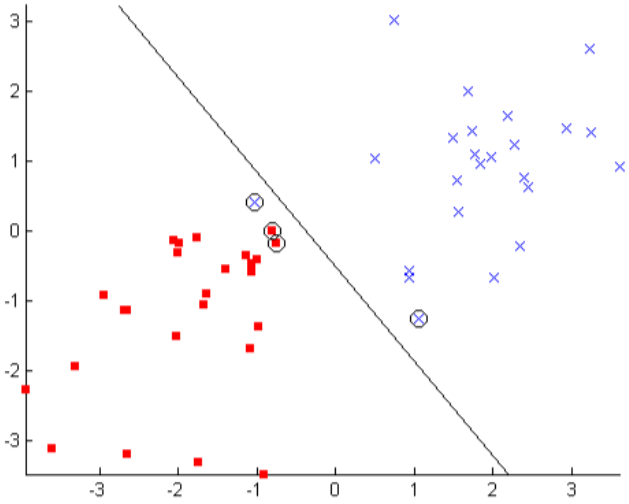


d

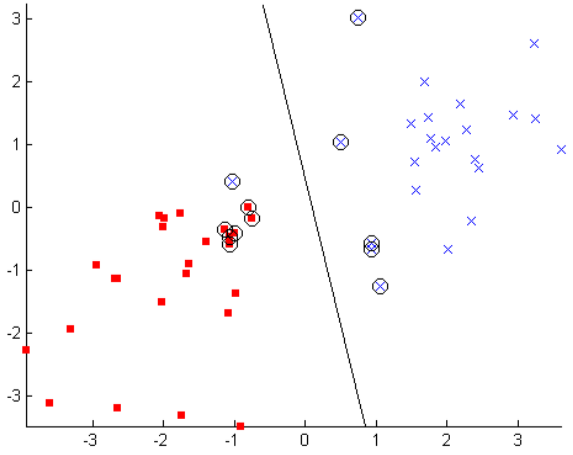


Support Vector Classifier

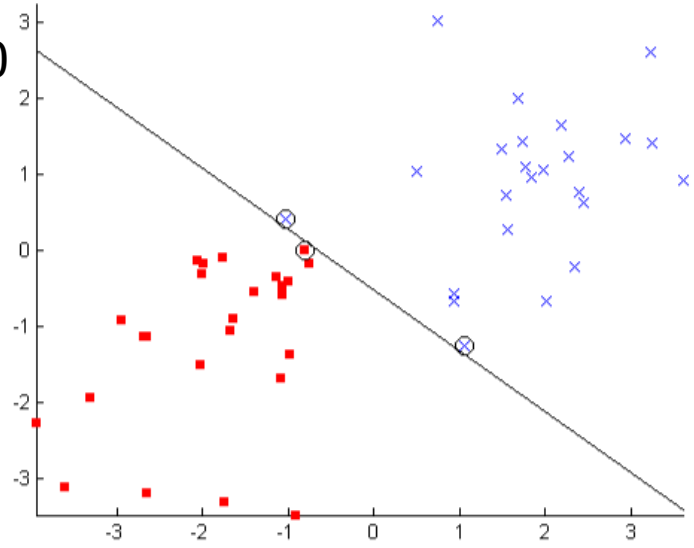
C=10



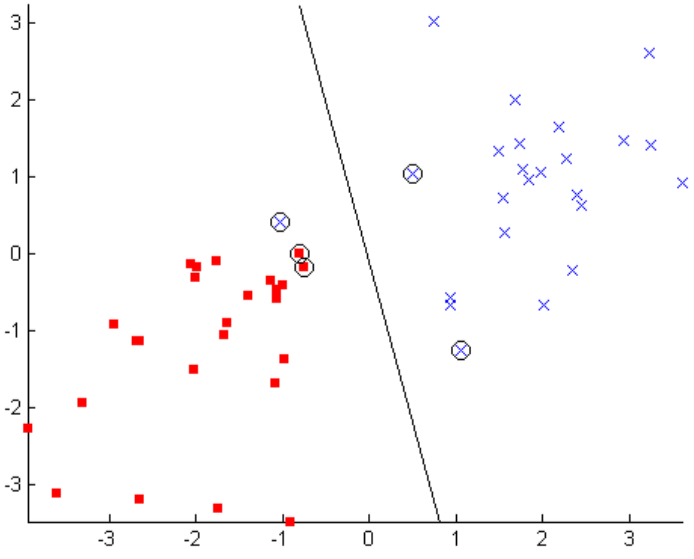
C=0.1



C=100



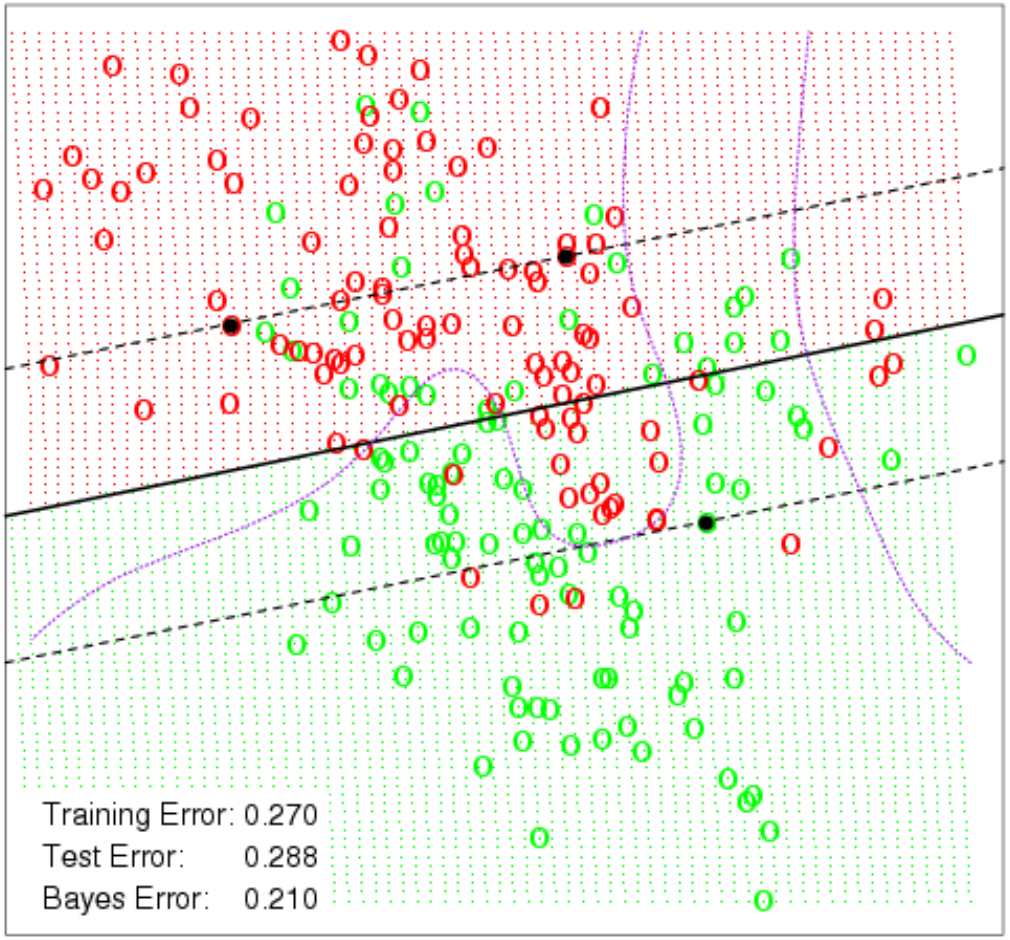
C=1



A Simulation Example With A Small Constant

The distance between the dashed line and the solid line represents the margin.

The purple lines represent the Naive Bayes decision boundary

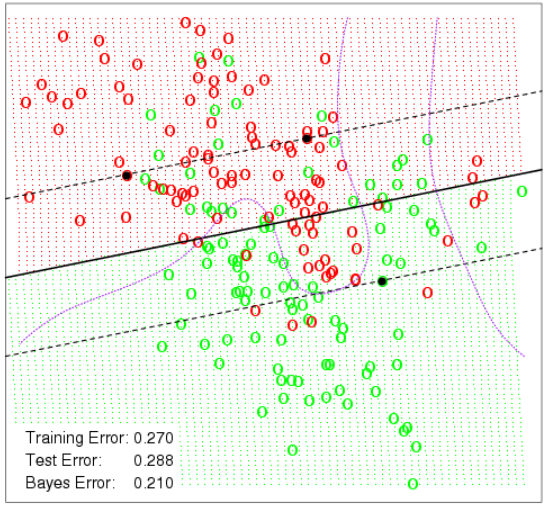


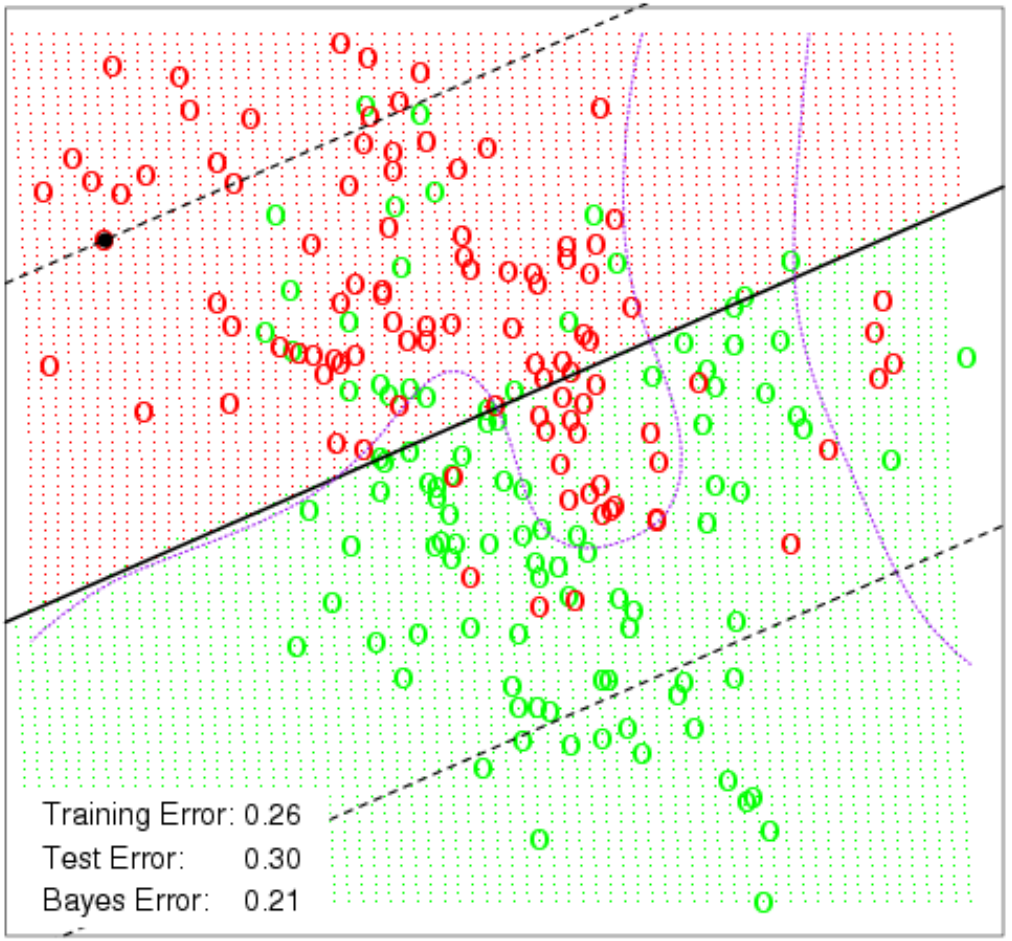
See also: https://www.csie.ntu.edu.tw/~cjlin/libsvm/

The Same Example With A Larger Constant

Using a larger constant allows for a greater margin and creates a slightly different classifier.

Notice, however, that the decision boundary must always be linear.





SVM Primal formulation

$$\underset{w,b}{\text{Minimize}} \; \frac{1}{2}\sum_{i=1}^{n} w_i^2 \quad \text{subject to} \quad y_i(\vec{w}\cdot\vec{x}_i + b) - 1 \geq 0 \quad \text{for i=1, ..., N}$$

Objective function — Constraints

- This is called "*primal formulation of linear SVMs*".
- It is a convex quadratic programming (QP) optimization problem with *n* variables (w_i, i= 1,…,n), where ***n*** is the number of features in the dataset.

Dual Optimization Problem

- The "*primal formulation of linear SVMs*" can be recast in the so-called "*dual form*" giving rise to "*dual formulation of linear SVMs*".
- It is also a convex quadratic programming problem but with *N* variables (α_i, $i = 1,\ldots,N$), where ***N*** is the number of samples.

Appendix C HML; ESLII – cap 12.2.1 page 420

Dual Optimization Problem

Maximize over α

$$\underset{\alpha}{\text{Maximize}} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{N} \alpha_i \alpha_j y_i y_j \vec{x}_i \cdot \vec{x}_j \quad \text{subject to} \quad \alpha_i \geq 0 \text{ and } \sum_{i=1}^{N} \alpha_i y_i = 0.$$

Objective function — Constraints

Then the *w*-vector is defined in terms of α_i:

$$\vec{w} = \sum_{i=1}^{N} \alpha_i y_i \vec{x_i}$$

Decision function

$$G(x)=\text{sign}(f(x)) = \text{sign}(b+ \vec{w} \cdot \vec{x}) = \text{sign}\left(b + \sum_{i=1}^{N} \alpha_i y_i \vec{x_i} \cdot \vec{x}\right)$$

Dual Optimization Problem

- No need to access original data, need to access only dot products.

Objective function: $\sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{N} \alpha_i \alpha_j y_i y_j \vec{x}_i \cdot \vec{x}_j$

Solution: $f(\vec{x}) = b + \sum_{i=1}^{N} \alpha_i y_i \vec{x}_i \cdot \vec{x}$

- Number of free parameters is bounded by the number of support vectors and not by the number of variables (beneficial for high-dimensional problems).

E.g., if a microarray dataset contains 20,000 genes and 100 patients, then need to find only up to 100 parameters!

Derivation of dual formulation

Minimize $\frac{1}{2}\sum_{i=1}^{n} w_i^2$ subject to $y_i(\vec{w}\cdot\vec{x}_i + b) - 1 \geq 0$ for $i = 1,\ldots,N$

Objective function

Constraints

Apply the method of Lagrange multipliers.

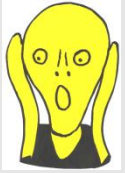
Define Lagrangian $\Lambda_P(\vec{w}, b, \vec{\alpha}) = \frac{1}{2}\sum_{i=1}^{n} w_i^2 - \sum_{i=1}^{N} \alpha_i (y_i(\vec{w}\cdot\vec{x}_i + b) - 1)$

a vector with *n* elements

a vector with *N* elements

We need to minimize this Lagrangian with respect to $\vec{w}, b$ and simultaneously require that the derivative with respect to $\vec{\alpha}$ vanishes , all subject to the constraints that $\alpha_i \geq 0$.

Derivation of dual formulation



If we set the derivatives with respect to $\vec{w}, b$ to 0, we obtain:

$$\frac{\partial \Lambda_P(\vec{w}, b, \vec{\alpha})}{\partial b} = 0 \Rightarrow \sum_{i=1}^{N} \alpha_i y_i = 0$$

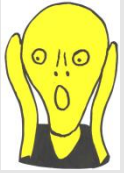
$$\frac{\partial \Lambda_P(\vec{w}, b, \vec{\alpha})}{\partial \vec{w}} = 0 \Rightarrow \vec{w} = \sum_{i=1}^{N} \alpha_i y_i \vec{x}_i$$

We substitute the above into the equation for $\Lambda_P(\vec{w}, b, \vec{\alpha})$ and obtain "*dual formulation of linear SVMs*":

$$\Lambda_D(\vec{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{N} \alpha_i \alpha_j y_i y_j \vec{x}_i \cdot \vec{x}_j$$

We seek to maximize the above Lagrangian with respect to $\vec{\alpha}$, subject to the constraints that $\alpha_i \geq 0$ and $\sum_{i=1}^{N} \alpha_i y_i = 0$.

Derivation of dual formulation

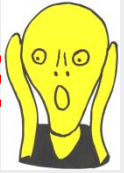


Because we are dealing with inequality constraints, there is an additional requirement: the solution must also satisfy the Karush-Kuhn-Tucker (KKT) conditions.

Moreover, the problem should satisfy some regularity conditions. Luckily for us, one of the regularity conditions is **Slater's condition.**

KKT conditions are also **sufficient** for the point to be primal and dual optimal, and there is zero duality gap

Karush-Kuhn-Tucker (KKT) conditions



- Stationarity condition:

$$\nabla_{\mathbf{w}}\mathcal{L} = \mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i = \mathbf{0}$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{m} \alpha_i y_i = 0$$

- Primal feasibility condition:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 \geq 0 \qquad \text{for all } i = 1, \ldots, m$$

- Dual feasibility condition:

$$\alpha_i \geq 0 \qquad \text{for all } i = 1, \ldots, m$$

- Complementary slackness condition:

$$\alpha_i[y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1] = 0 \qquad \text{for all } i = 1, \ldots, m$$

Inner products and support vectors

The linear support vector classifier can be represented as

$$f(\vec{x}) = b + \sum_{i=1}^{N} \alpha_i y_i \overrightarrow{x_i} \cdot \vec{x}$$ — N parameters

To estimate the parameters $\alpha_1, \ldots, \alpha_N$ and b, all we need are the inner products $\overrightarrow{x_i} \cdot \overrightarrow{x_j}$ between all pairs of training observations.

Inner products and support vectors

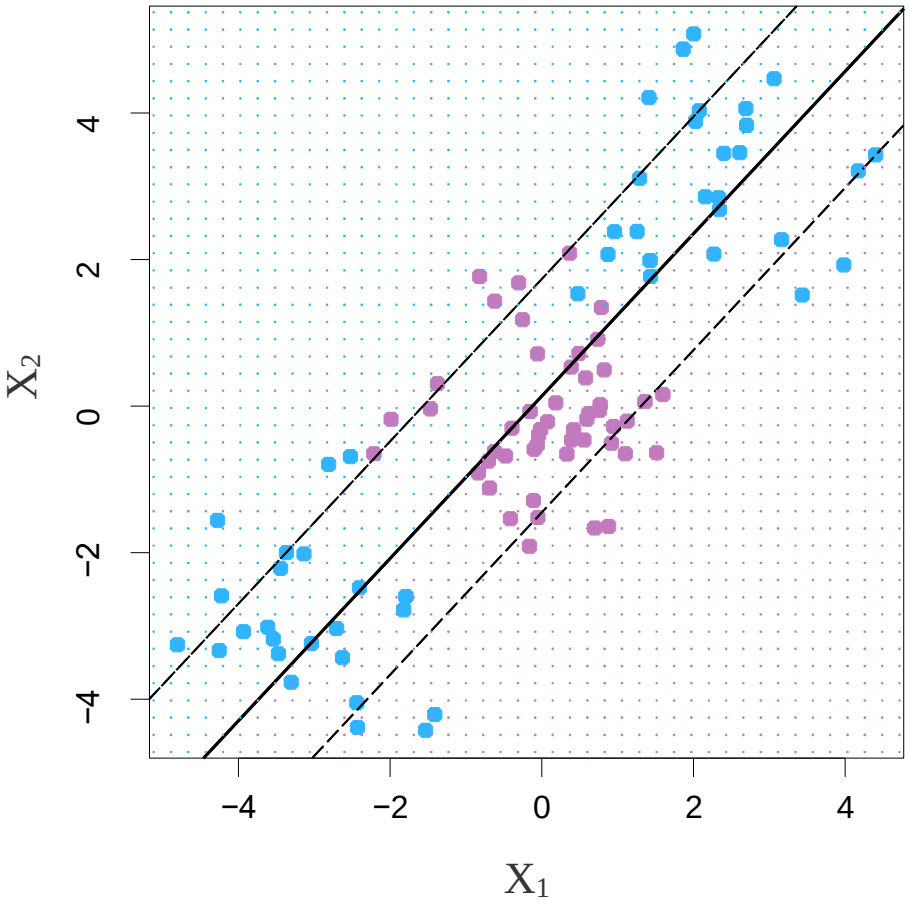
It turns out that most of the α_i can be zero:

$$f(\vec{x}) = b + \sum_{S} \alpha_i y_i \overrightarrow{x_i} \cdot \vec{x}$$

S is the support set of indices i such that $\alpha_i > 0$.

Kernel Functions

Linear boundary can fail



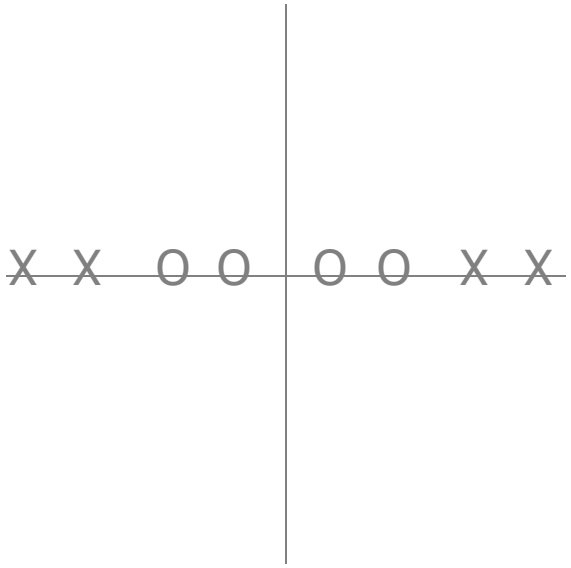
Sometime a linear boundary simply won't work, no matter what value of *C*

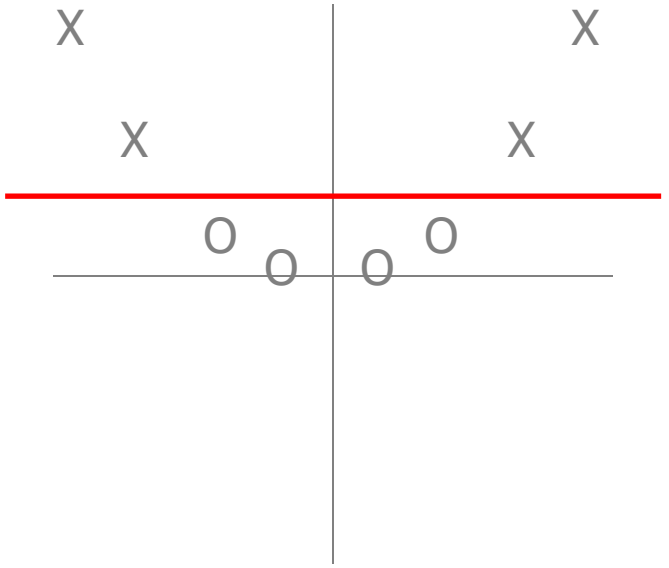
The example on the left is such a case.

What to do?

Feature Expansion

Having the right features (x) is crucial



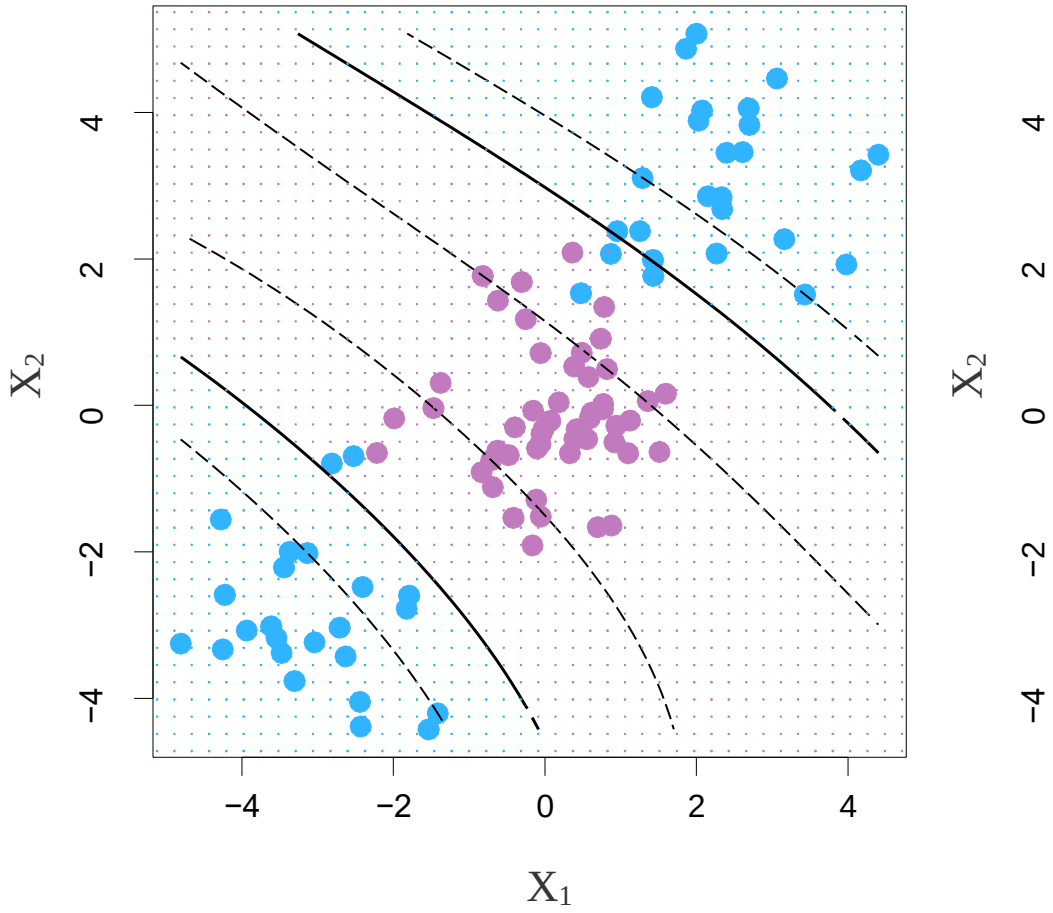


Cubic Polynomials

Here we use a basis expansion of cubic polynomials

From 2 variables to 9

The support-vector classifier in the enlarged space solves the problem in the lower-dimensional space



$$w_0 + w_1 X_1 + w_2 X_2 + w_3 X_1^2 + w_4 X_2^2 + w_5 X_1 X_2 + w_6 X_1^3 + w_7 X_2^3 + w_8 X_1 X^2 + w_9 X_1^2 X_2 = 0$$

Feature Expansion

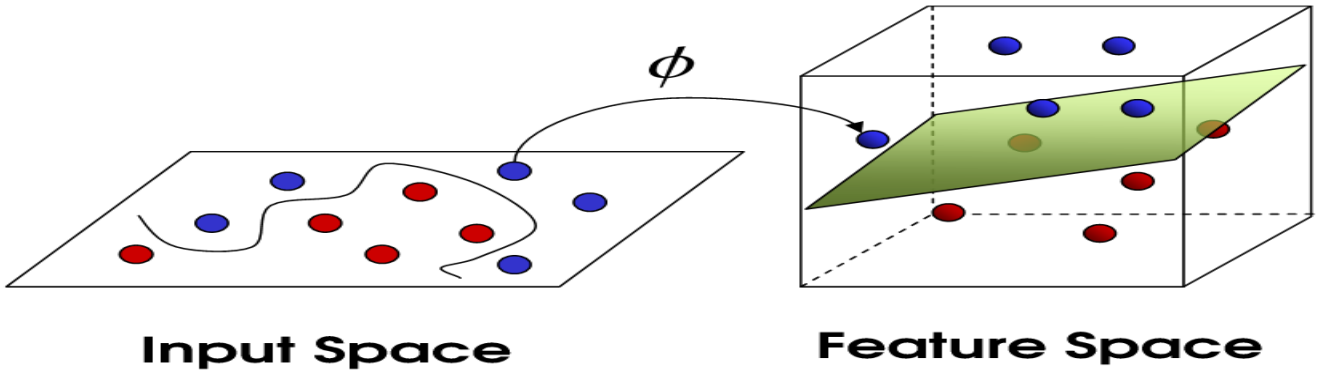
- Enlarge the space of features by including transformations; e.g. $X_1^2, X_1^3, X_1X_2, \ldots$ Hence go from a p-dimensional space to a $p' > p$ dimensional space.
- Fit a support-vector classifier in the enlarged space.
- This results in non-linear decision boundaries in the original space.

Example: Suppose we use $(X_1, X_2, X_1^2, X_2^2, X_1X_2)$ instead of just (X_1, X_2). Then the decision boundary would be of the form

$$w_0 + w_1X_1 + w_2X_2 + w_3X_1^2 + w_4X_2^2 + w_5X_1X_2 = 0$$

This leads to nonlinear decision boundaries in the original space (quadratic conic sections).

Mapping into a New Feature Space



$$\Phi : x \rightarrow X = \Phi(x)$$

$$\Phi(x_1,x_2) = (x_1,x_2,x_1^2,x_2^2,x_1x_2)$$

Rather than run SVM on x_i, run it on $\Phi(x_i)$

Find non-linear separator in input space

What if $\Phi(x_i)$ is really big?

Image from http://web.engr.oregonstate.edu/~afern/classes/cs534/

Nonlinearities and Kernels

- Polynomials (especially high-dimensional ones) get wild rather fast.

- There is a more elegant and controlled way to introduce nonlinearities in support-vector classifiers — through the use of kernels.

Kernel Functions

Informally a kernel function is a special similarity measure $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{R}$ between patterns lying in some arbitrary domain $\mathcal{X}$, which represents a dot product, denoted by $\langle \cdot, \cdot \rangle$, in some Hilbert space.

kernel trick: Instead of performing the expensive transformation step explicitly, the kernel can be calculated directly, thus performing the feature transformation only implicitly.

Kernel Functions

Definition

Let $\mathcal{X}$ be a set. A symmetric function $k : \mathcal{X} \times \mathcal{X} \to \boldsymbol{R}$ is a positive definite **kernel function** on $\mathcal{X}$ iff $\forall n \in \boldsymbol{N}$, $\forall x_1, \ldots, x_n \in \mathcal{X}$, and $\forall c_1, \ldots, c_n \in \boldsymbol{R}$

$$\sum_{i,j \in \{1,\ldots,n\}} c_i c_j k(x_i, x_j) \geq 0$$

Proposition (Smola & Scholkopf 2001)

Kernel functions are closed under sum, direct sum, multiplication by a scalar, tensor product, zero extension, pointwise limits, and exponentiation

Kernel Functions

Theorem (Mercer's property)

For any positive definite kernel function $k \in \mathbb{R}^{\mathcal{X}\times\mathcal{X}}$, *there exists a mapping* $\phi \in \mathcal{H}^{\mathcal{X}}$ *into the feature space* $\mathcal{H}$ *equipped with the inner product* $\langle\cdot,\cdot\rangle_{\mathcal{H}}$, *such that:*

$$\forall u, v \in \mathcal{X}, \qquad k(u,v) = \langle \phi(u), \phi(v) \rangle_{\mathcal{H}}$$

Kernel Functions

Original data $\vec{x}$ (in input space)

$$f(x) = sign(\vec{w} \cdot \vec{x} + b)$$

$$\vec{w} = \sum_{i=1}^{N} \alpha_i y_i \vec{x}_i$$

Data in a higher dimensional feature space $\Phi(\vec{x})$

$$f(x) = sign(\vec{w} \cdot \Phi(\vec{x}) + b)$$

$$\vec{w} = \sum_{i=1}^{N} \alpha_i y_i \Phi(\vec{x}_i)$$

$$f(x) = sign(\sum_{i=1}^{N} \alpha_i y_i \Phi(\vec{x}_i) \cdot \Phi(\vec{x}) + b)$$

$$f(x) = sign(\sum_{i=1}^{N} \alpha_i y_i K(\vec{x}_i, \vec{x}) + b)$$

Therefore, we do not need to know Φ explicitly, we just need to define function $K(\cdot, \cdot)$: $\mathcal{X} \times \mathcal{X} \rightarrow \mathcal{R}$.

Kernel Functions

What does it *mean* to be a kernel?

A kernel is a dot product in ***some*** feature space:

$K(x_1,x_2) = < \phi(x_1), \phi(x_2) >$ for some ϕ

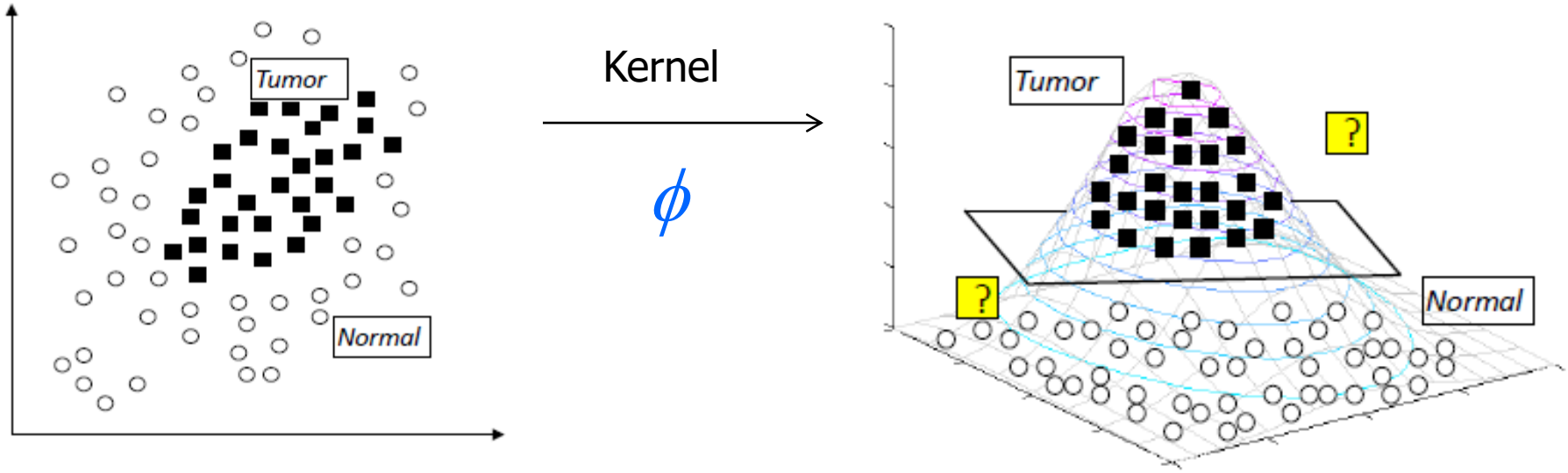
For all x_1 and x_2 in the input space, the kernel function $K(x_1,x_2)$ can be expressed as an inner product in another space *H*.

$\phi : X \rightarrow H$ is called "feature map"

Kernel Functions

Data is not linearly separable in the input space

Data is linearly separable in the feature space obtained by a kernel



Kernel Functions

"Given an algorithm which is formulated in terms of a positive definite kernel K_1, one can construct an alternative algorithm by replacing K_1 with another positive definite kernel K_2"

- SVMs can use the kernel trick

Using a Different Kernel in the Dual Optimization Problem

For example, using the polynomial kernel with d = 4 (including lower-order terms).

Maximize over α

$$W(\alpha) = \Sigma_i \alpha_i - 1/2 \Sigma_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$$

Subject to

$$\alpha_i \geq 0$$

$$\Sigma_i \alpha_i y_i = 0$$

$(\langle x_i, x_j \rangle + 1)^4$

These are kernels!

So by the kernel trick we just replace them!

Decision function

$$f(x) = \text{sign}(\Sigma_i \alpha_i y_i \langle x, x_i \rangle + b)$$

$(\langle x_i, x_j \rangle + 1)^4$

A Few Good Kernels

$$K(\vec{x}_i, \vec{x}_j) = \vec{x}_i \cdot \vec{x}_j$$ Linear kernel

$$K(\vec{x}_i, \vec{x}_j) = \exp(-\gamma \left\| \vec{x}_i - \vec{x}_j \right\|^2)$$ Gaussian kernel

$$K(\vec{x}_i, \vec{x}_j) = \exp(-\gamma \left\| \vec{x}_i - \vec{x}_j \right\|)$$ Exponential kernel

$$K(\vec{x}_i, \vec{x}_j) = (p + \vec{x}_i \cdot \vec{x}_j)^q$$ Polynomial kernel

$$K(\vec{x}_i, \vec{x}_j) = (p + \vec{x}_i \cdot \vec{x}_j)^q \exp(-\gamma \left\| \vec{x}_i - \vec{x}_j \right\|^2)$$ Hybrid kernel

$$K(\vec{x}_i, \vec{x}_j) = \tanh(k\vec{x}_i \cdot \vec{x}_j - \delta)$$ Sigmoidal

The Polynomial Kernel (d=2, p=0)

$K(x_1,x_2) = < x_1, x_2 >^2$

Where $x_1 = (x_{11}, x_{12})$, $x_2 = (x_{21}, x_{22})$

Using:

$$\Phi(x_1) = (x_{11}^2, x_{12}^2, \sqrt{2}x_{11}\, x_{12})$$

$$\Phi(x_2) = (x_{21}^2, x_{22}^2, \sqrt{2}\, x_{21}\, x_{22})$$

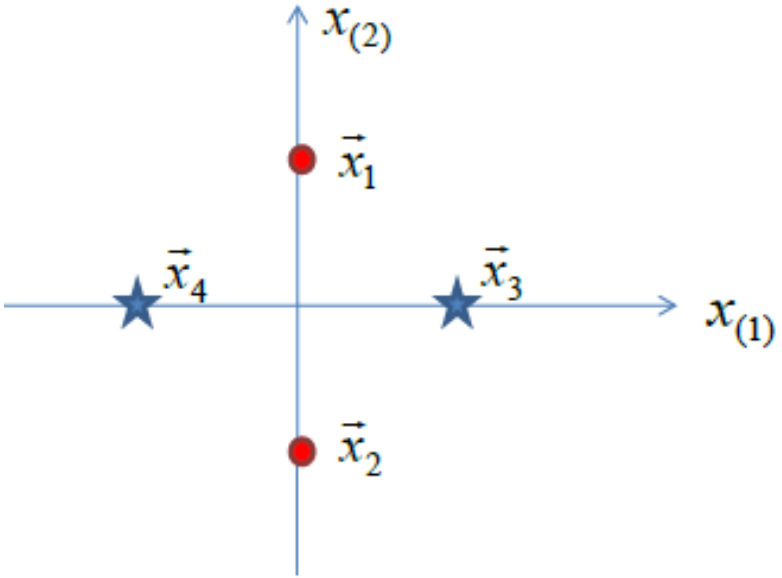
We have

$$K(x_1,x_2) = < \Phi(x_1), \Phi(x_2) >= (x_{11}^2\, x_{21}^2 + x_{12}^2 x_{22}^2 + 2x_{11}\, x_{12}\, x_{21}\, x_{22}) = (x_{11}x_{21} + x_{12}x_{22})^2 = (< x_1, x_2 >)^2 = < x_1, x_2 >^2$$

The Polynomial Kernel (d=2)

Therefore, the explicit mapping is

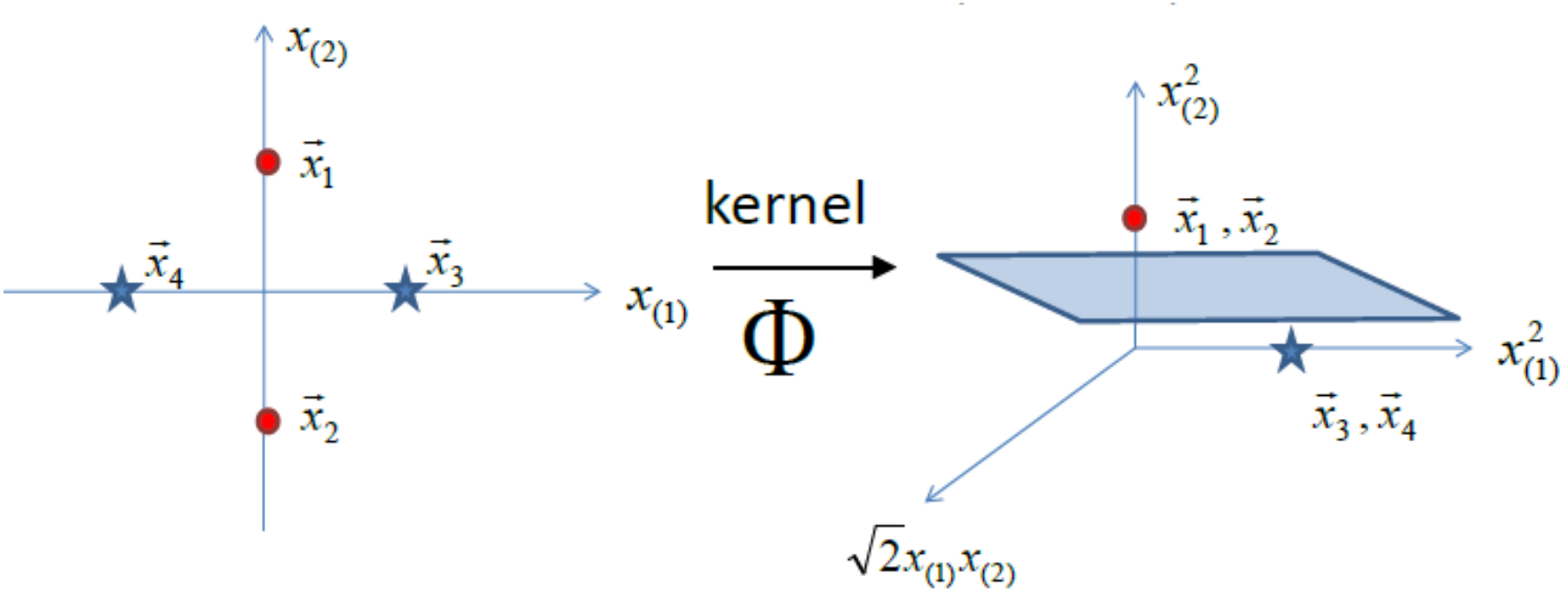
$$\Phi(\vec{x}) = \begin{pmatrix} x_{(1)}^2 \\ \sqrt{2}x_{(1)}x_{(2)} \\ x_{(2)}^2 \end{pmatrix}$$



The Polynomial Kernel (d=2)

Therefore, the explicit mapping is

$$\Phi(\vec{x}) = \begin{pmatrix} x_{(1)}^2 \\ \sqrt{2}x_{(1)}x_{(2)} \\ x_{(2)}^2 \end{pmatrix}$$



The Polynomial Kernel (d=2, p=1)

$K(x_1,x_2) = (< x_1, x_2 > +1)^2$

Where $x_1 = (x_{11}, x_{12})$, $x_2 = (x_{21}, x_{22})$

Using:

$$\Phi(x_1) = (x_{11}^2, x_{12}^2, \sqrt{2}x_{11}, \sqrt{2}x_{12}, \sqrt{2}x_{11}x_{12}, 1)$$

$$\Phi(x_2) = (x_{21}^2, x_{22}^2, \sqrt{2}x_{21}, \sqrt{2}x_{22}, \sqrt{2}x_{21}x_{22}, 1)$$

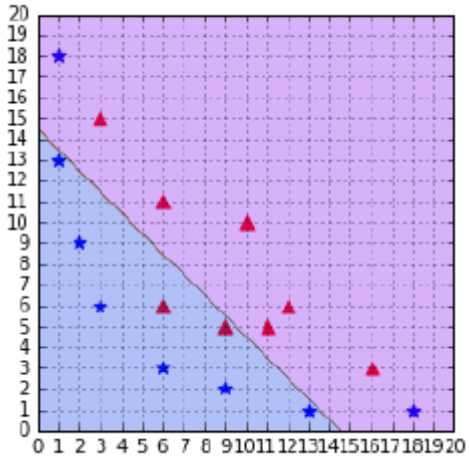
We have

$$K(x_1,x_2) = < \Phi(x_1), \Phi(x_2) >= (x_{11}^2 x_{21}^2 + x_{12}^2 x_{22}^2 + 2x_{11}x_{21} + 2x_{12}x_{22} + 2x_{11}x_{12}x_{21}x_{22} + 1) = (x_{11}x_{21} + x_{12}x_{22} + 1)^2 = (< x_1, x_2 > +1)^2$$

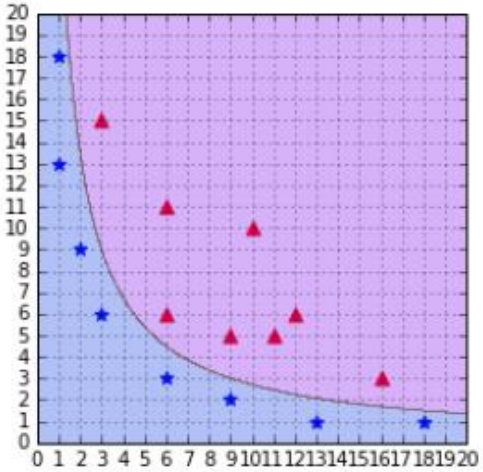
The Polynomial Kernel

- A polynomial kernel with a degree of 1 and no constant is simply the linear kernel .
- When you increase the degree of a polynomial kernel, the decision boundary will become more complex and will have a tendency to be influenced by individual data examples .
- Using a high-degree polynomial is dangerous because you can often achieve better performance on your training set, but it leads to what is called **overfitting**
- See video: http://www.youtube.com/watch?v=3liCbRZPrZA

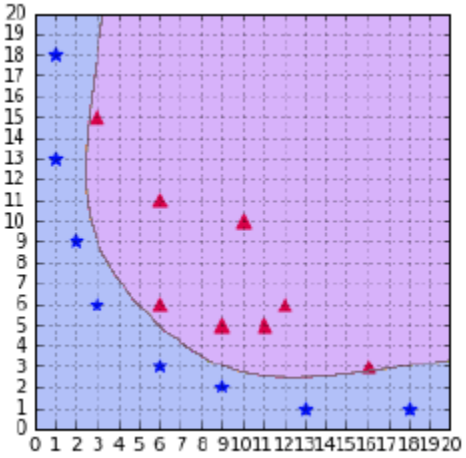
The Polynomial Kernel



d=1



d=2

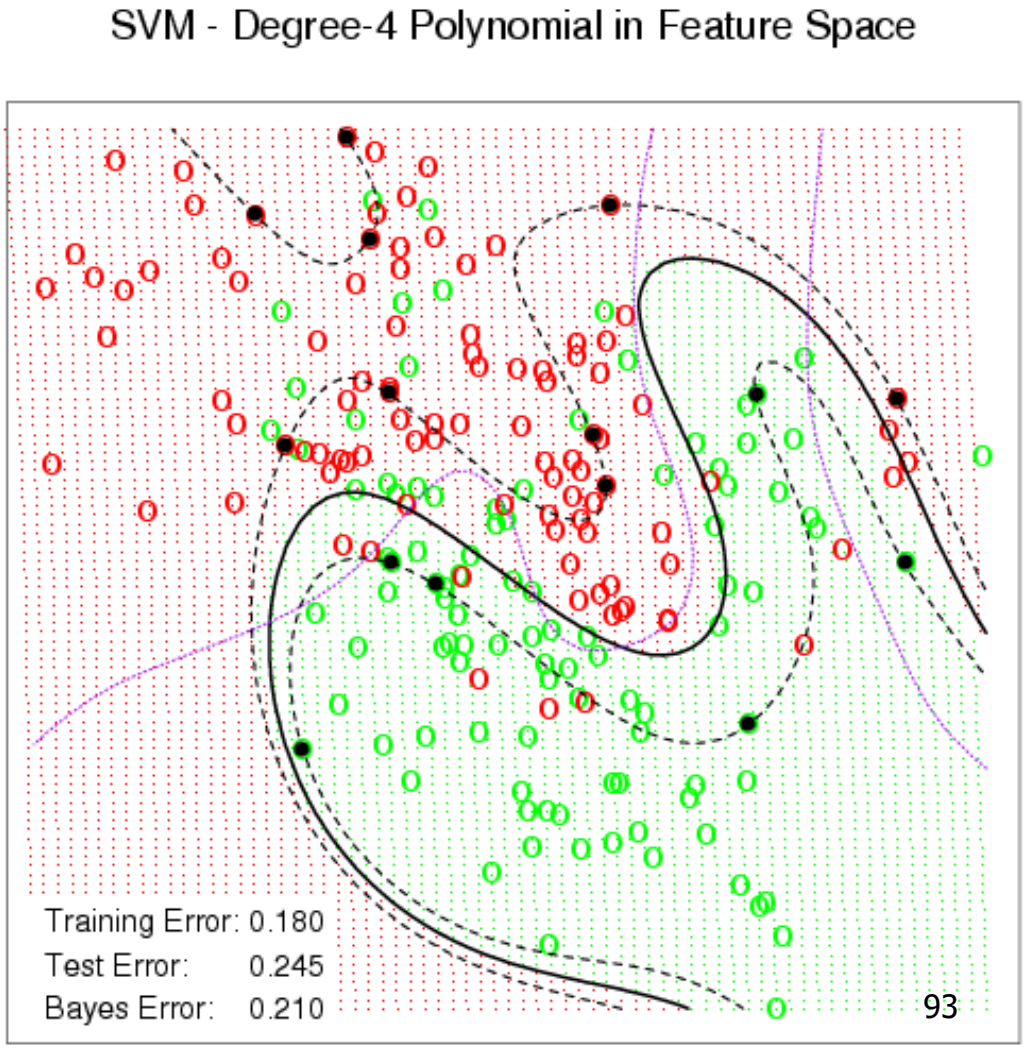


d=6

Note: Using a high-degree polynomial kernel will often lead to overfitting

Polynomial Kernel On Sim Data

Using a polynomial kernel we now allow SVM to produce a non-linear decision boundary.



Gaussian (or radial basis) kernel

Consider Gaussian kernel: $K(\vec{x}, \vec{x}_j) = \exp(-\gamma \left\| \vec{x} - \vec{x}_j \right\|^2)$

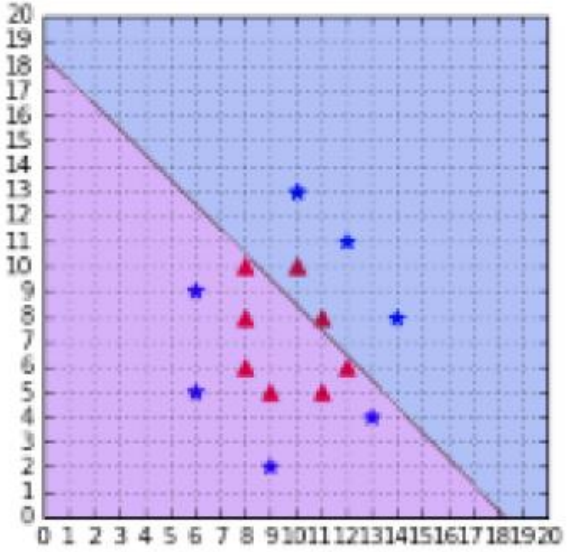
It depends only on the distance from the origin or from some point.

RBF kernel returns the result of a dot product performed in $\mathbb{R}^{\infty}$

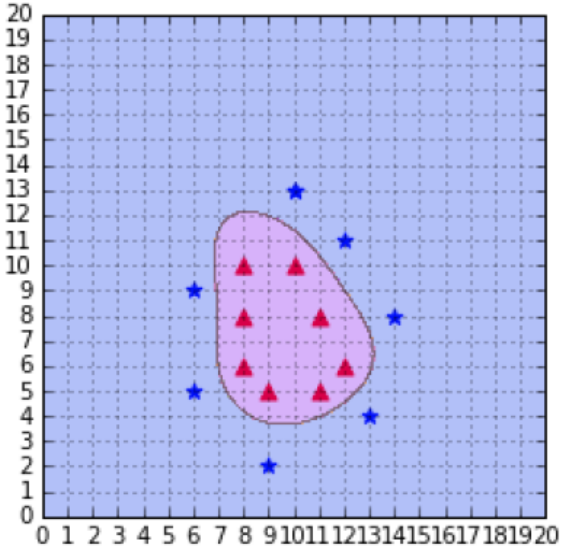
When gamma is too small the model behaves like a linear SVM.

When gamma is too large, the model is too heavily influenced by each support vector .

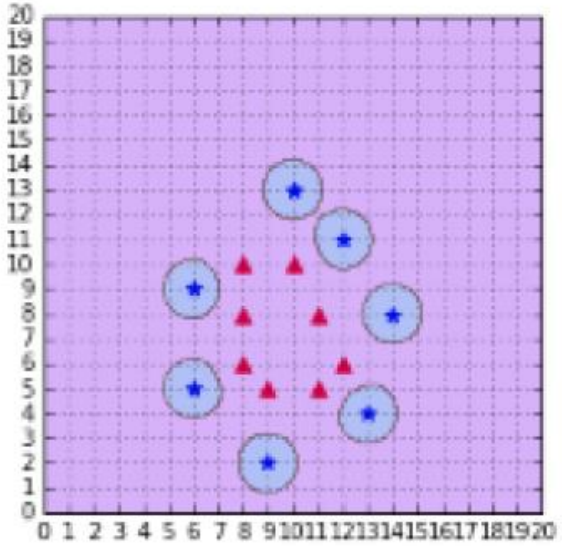
Gaussian (or radial basis) kernel



$\gamma = 1e-5$



$\gamma = 0.1$

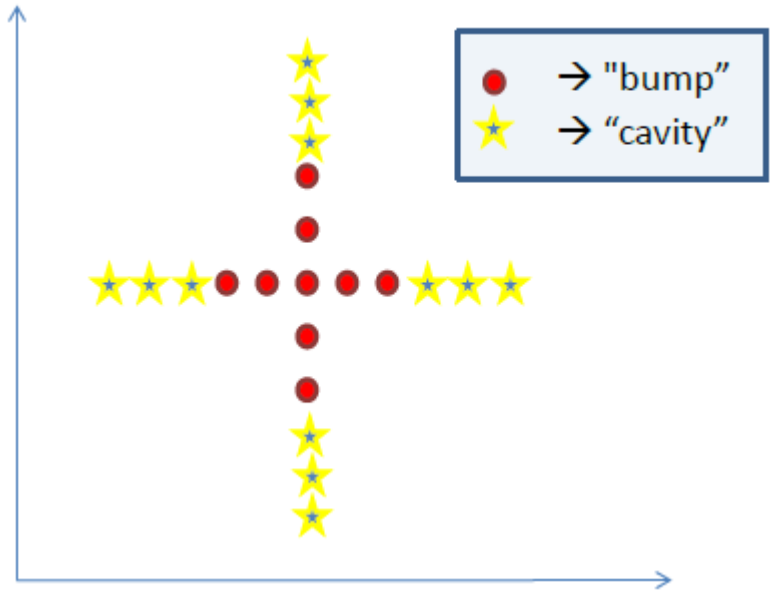


$\gamma = 2$

Understanding the Gaussian (or radial basis) kernel

$$K(\vec{x}, \vec{x}_j) = \exp(-\gamma \left\| \vec{x} - \vec{x}_j \right\|^2)$$

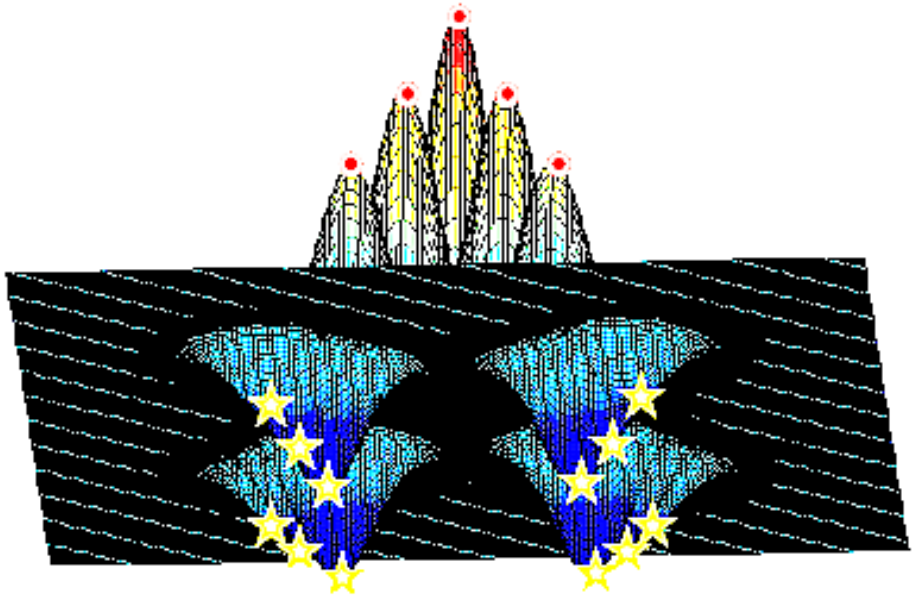
Geometrically, this is a "bump" or "cavity" centered at the training data point $\overrightarrow{x_j}$:

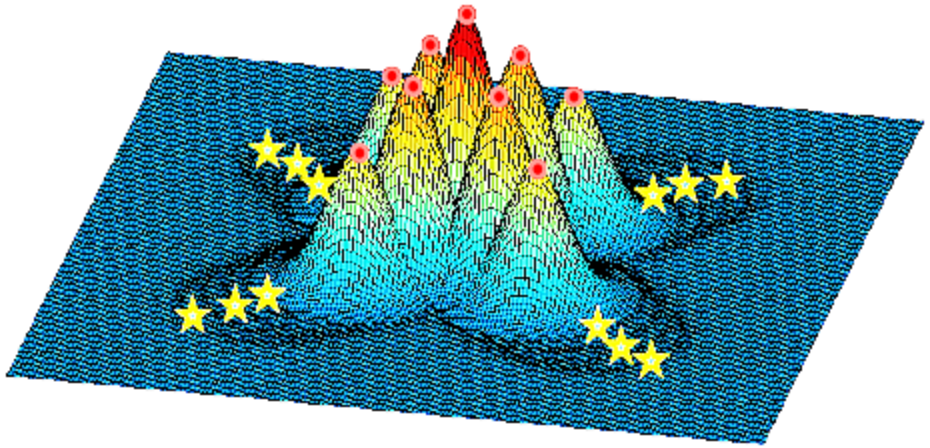


The resulting mapping function is a **combination** of bumps and cavities.

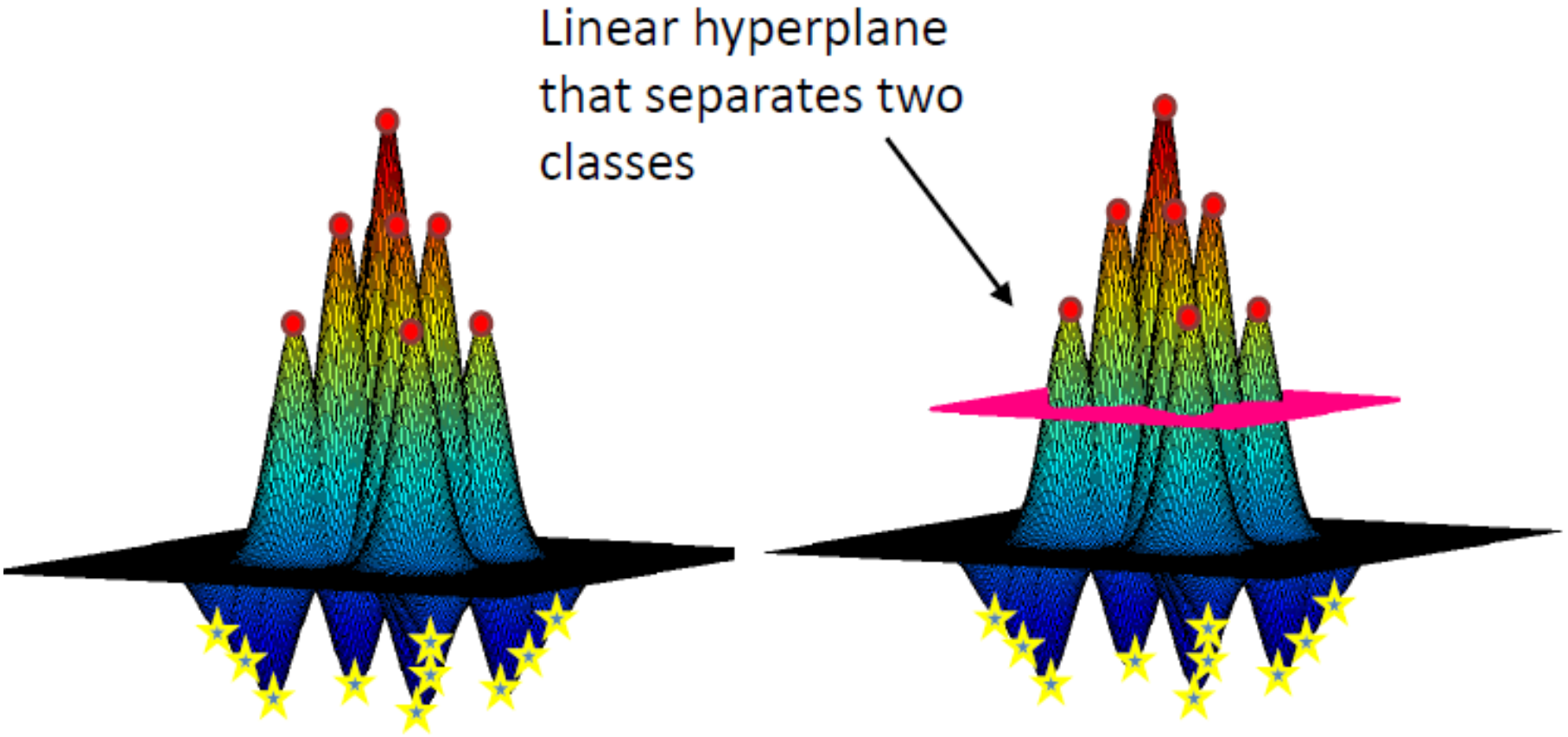
Understanding the Gaussian kernel

Several more views of the data is mapped to the feature space by Gaussian kernel





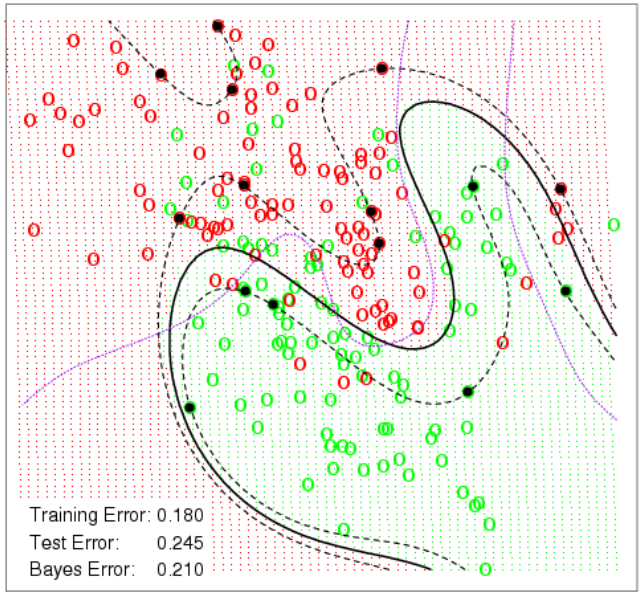
Understanding the Gaussian kernel



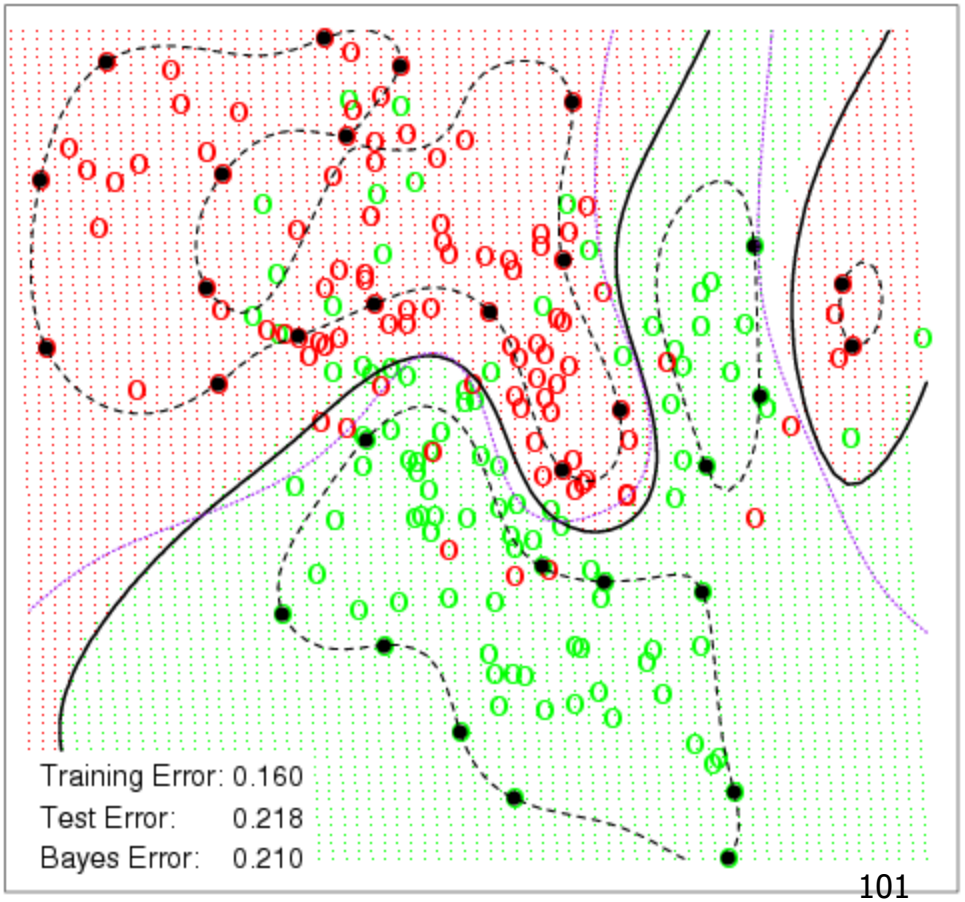
Radial Basis Kernel

Using a Radial Basis Kernel you get an even lower error rate.

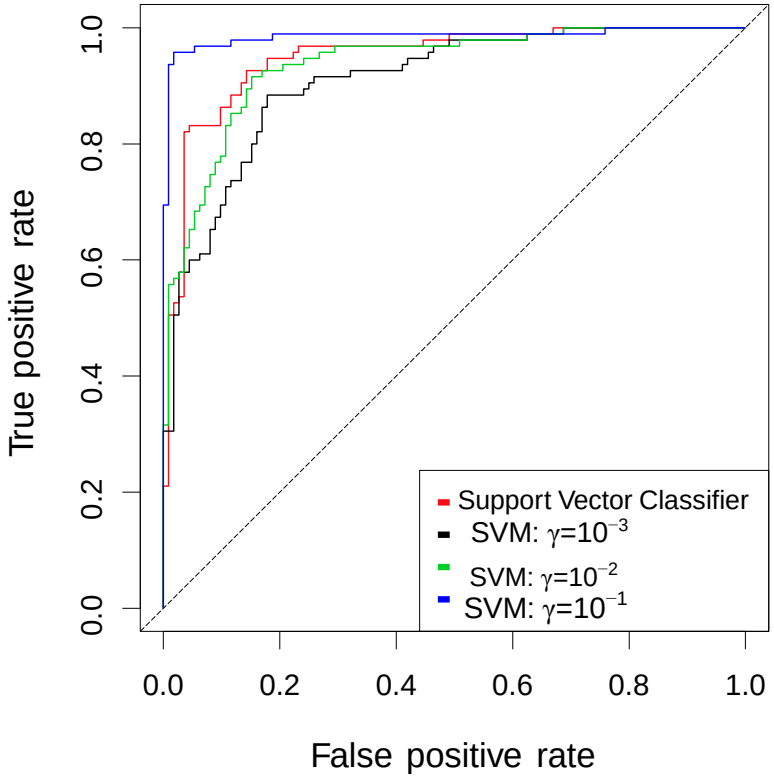
SVM - Degree-4 Polynomial in Feature Space



SVM - Radial Kernel in Feature Space

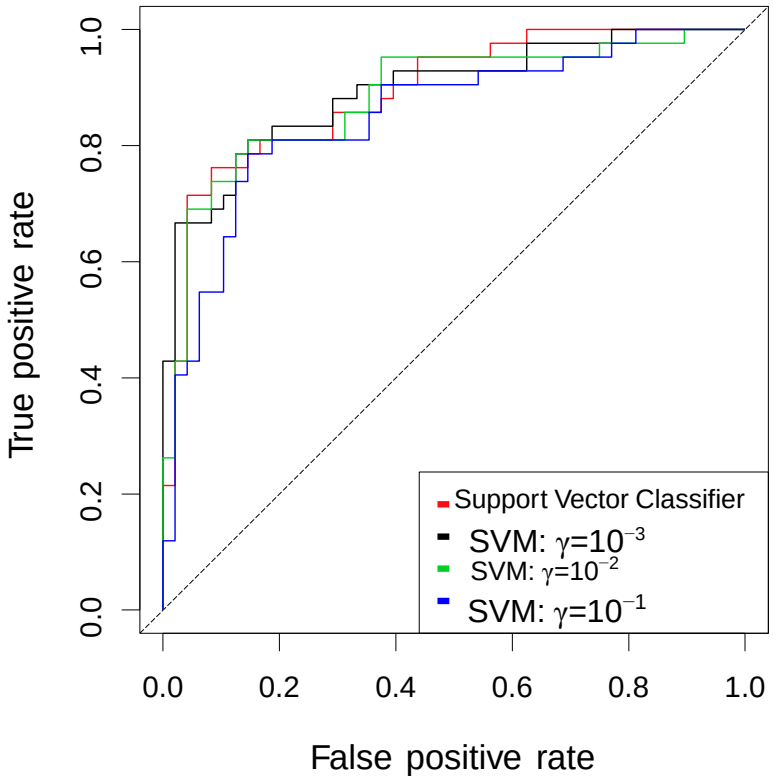


Example: Heart Data



ROC curve is obtained by changing the threshold 0 to threshold t in $\hat{f}(X) > t$, and recording false positive and true positive rates as t varies. Here we see ROC curves on **training data**.

Example: Heart Data



Here we see ROC curves on **test data**.

Exotic Kernels

- Research on kernels has been prolific, and there are now a lot of kernels available.
- Some of them are specific to a domain, such as the string kernel, which can be used when working with text, or graph kernels.
- If you want to discover more kernels, this article from César Souza describes 25 kernels.

Which kernel should I use?

- The recommended approach is to try a RBF kernel first, because it usually works well.
- However, it is good to try the other types of kernels if you have enough time to do so.
- **A kernel is a measure of the similarity between two vectors**, so that is where domain knowledge of the problem at hand may have the biggest impact.
- Building a custom kernel can also be a possibility, but it requires that you have a good mathematical understanding of the theory behind kernels.

SVMs with more than 2 classes

SVMs: more than 2 classes?

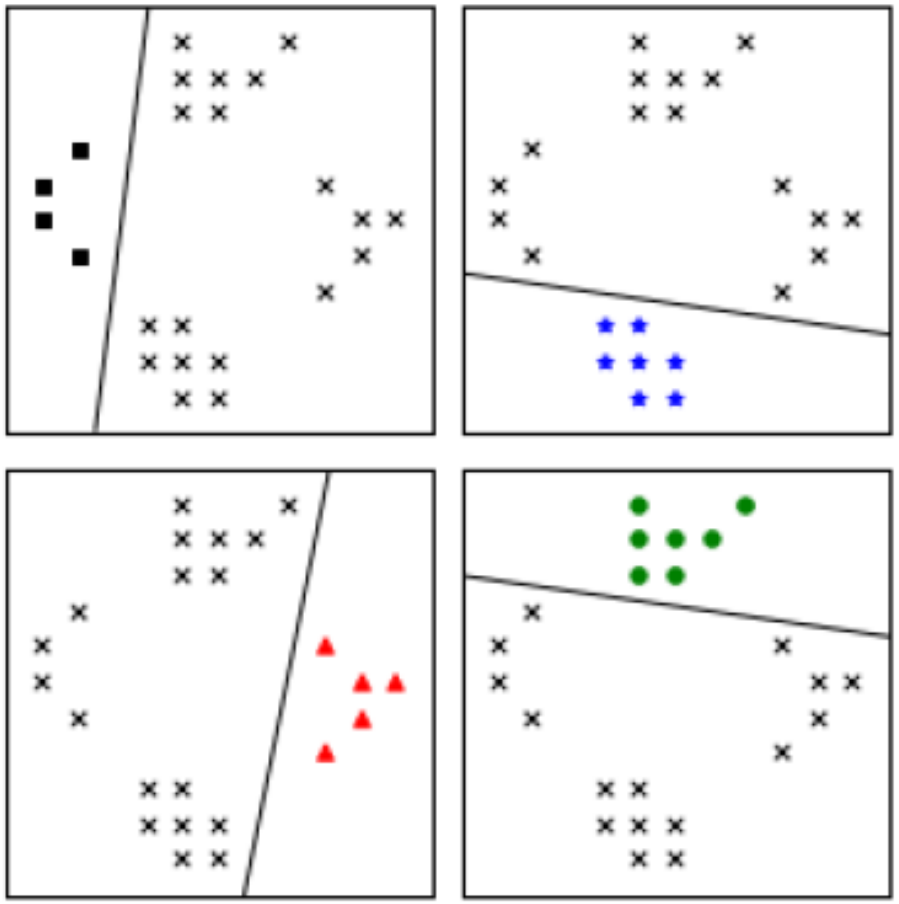
- The SVM as defined works for $K = 2$ classes.
- What do we do if we have $K > 2$ classes?
- There are several approaches that allow SVMs to work for multi-class classification.

SVMs: more than 2 classes?

OVA: One versus All

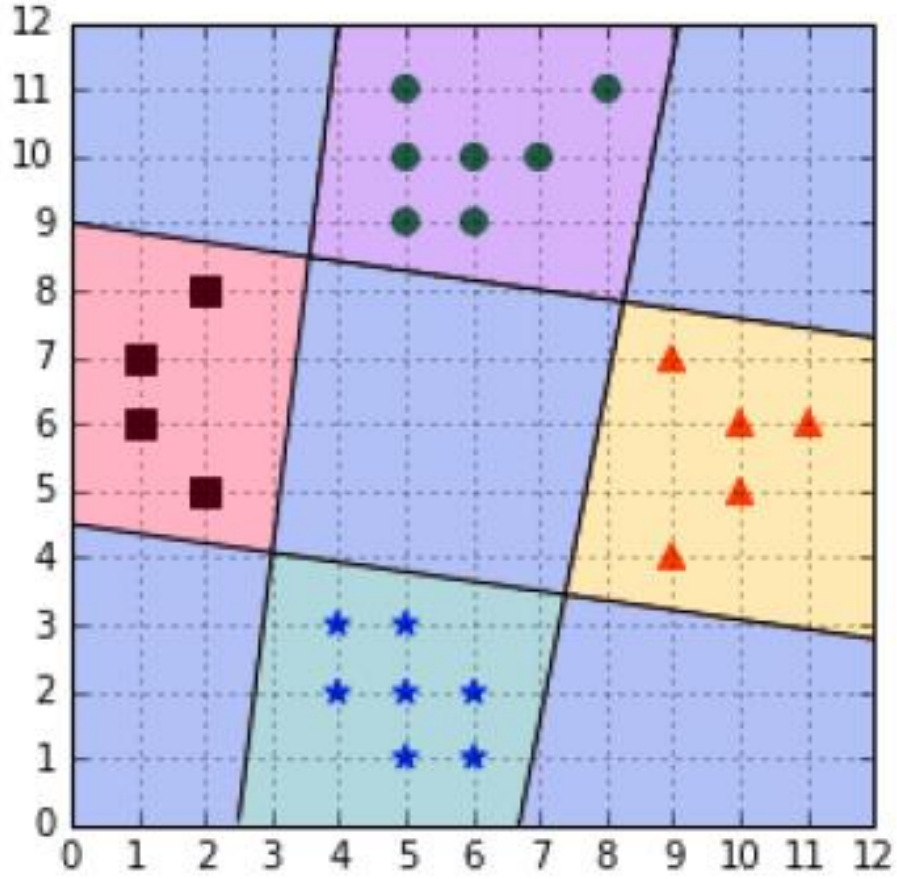
- In order to classify K classes, we construct K different binary classifiers. For a given class, the positive examples are all the points in the class, and the negative examples are all the points not in the class.
- In order to make a new prediction, we use each classifier and predict the class of the classifier if it returns a positive answer.
- However, this can give inconsistent results because a label is assigned to multiple classes simultaneously or to none.

OVA



The One-Versus-All approach creates one classifier per class

OVA

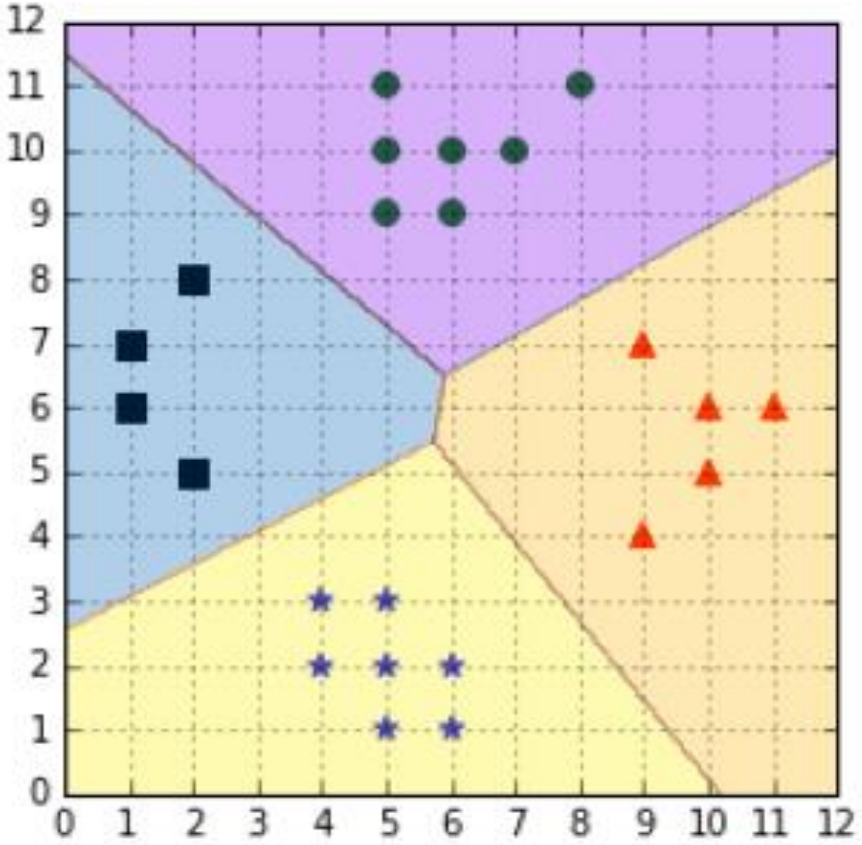


One-Versus-All leads to ambiguous decisions

OVA

- Vladimir Vapnik (1998) suggested to use the class of the classifier for which the value of the decision function is the maximum.
- This method returns a real value that will be positive if the example is on the correct side of the classifier, and negative if it is on the other side.
- This approach will choose the class of the hyperplane the closest to the example when all classifiers disagree.
- For instance, the example point (6,4) in the previous Figure will be assigned the blue star class.

OVA



Applying a simple heuristic avoids the ambiguous decision problem

OVA Issues

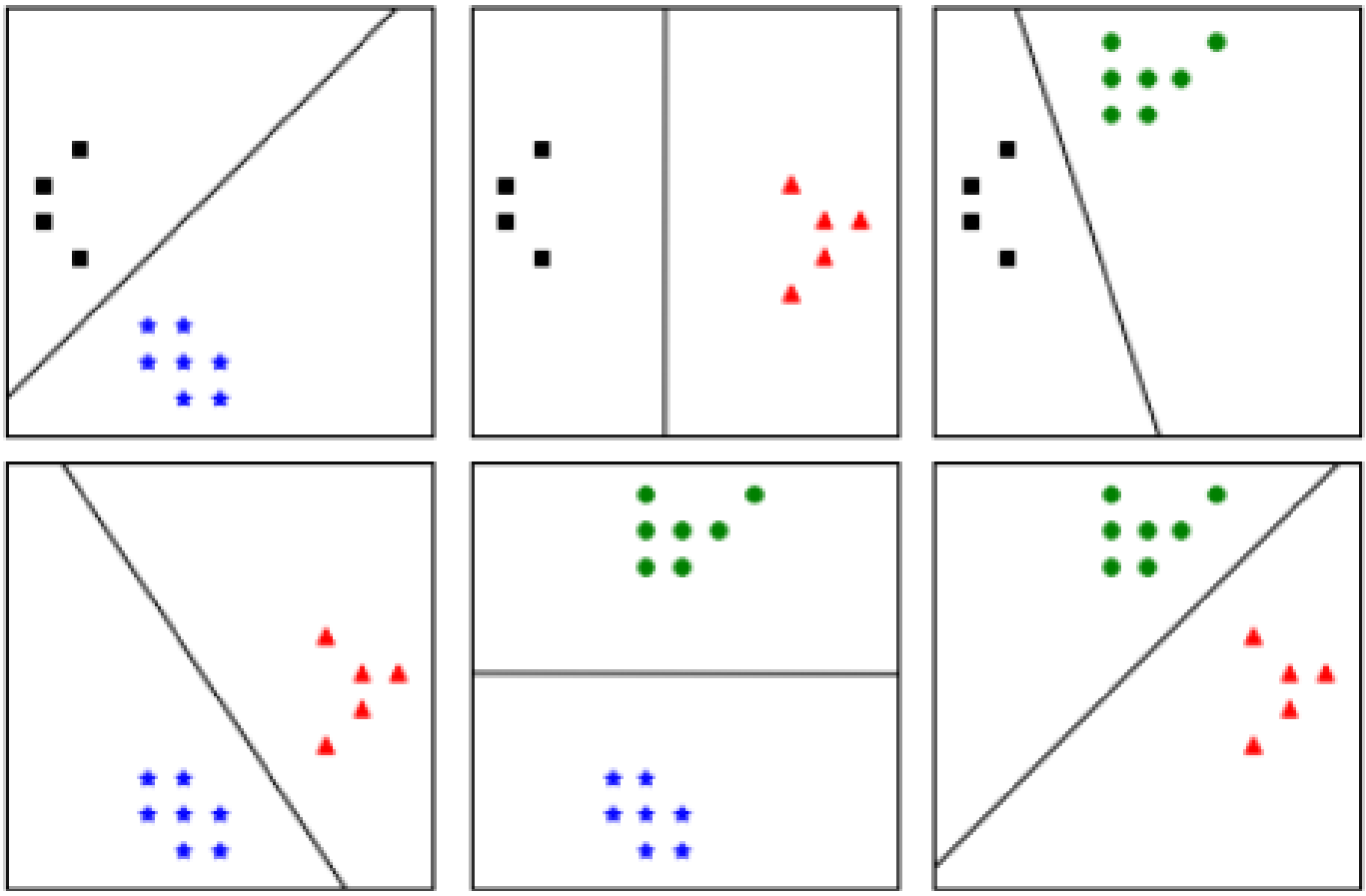
- Different classifiers were trained on different tasks, so there is no guarantee that the quantities returned by the **decision function** have the same scale.
- Training sets are imbalanced: for a problem with 100 classes, each having 10 examples, each classifier will be trained with 10 positive examples and 990 negative examples. Thus, the negative examples will influence the decision boundary greatly.

OVA remains a popular method for multi-class classification because it is easy to implement and understand.

SVMs: more than 2 classes?

- One-versus-One: train one classifier per pair of classes, which leads to K(K-1)/2 classifiers for K classes. Each classifier is trained on a subset of the data and produces its own decision boundary.
- Predictions are made using a simple **voting strategy**.
- Each example we wish to predict is passed to each classifier, and the predicted class is recorded.
- The class having the most votes is assigned to the example.

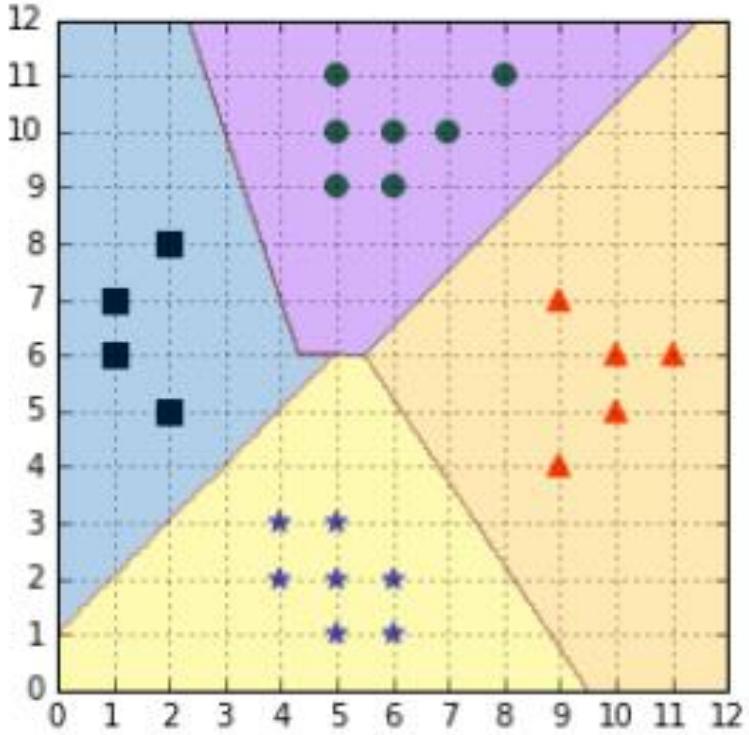
OVO



One-versus-one construct with one classifier for each pair of classes

OVO

- Issues: two classes can have an identical number of votes.
- it has been suggested that selecting the one with the smaller index might be a viable (while probably not the best) strategy



SVMs: more than 2 classes?

The SVM as defined works for $K = 2$ classes. What do we do if we have $K > 2$ classes?

OVA One versus All. Fit K different 2-class SVM classifiers $\hat{f}_k(x)$, $k = 1, \ldots, K$; each class versus the rest. Classify x^* to the class for which $\hat{f}_k(x^*)$ is largest.

OVO One versus One. Fit all $\binom{K}{2}$ pairwise classifiers $\hat{f}_{k`}(x)$. Classify x^* to the class that wins the most pairwise competitions.

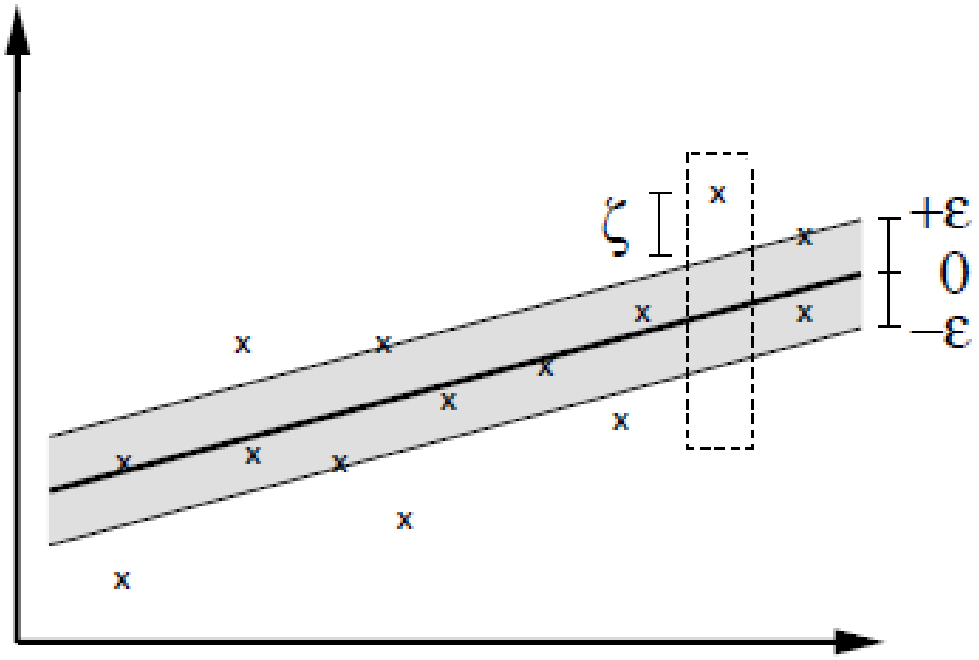
Which to choose? If K is not too large, use OVO.

SVM Regression

- Maximum margin hyperplane only applies to classification
- However, idea of support vectors and kernel functions can be used for regression
- Basic method is the same as in linear regression: want to minimize error
- Difference A: ignore errors smaller than e and use absolute error instead of squared error
- Difference B: simultaneously aim to maximize flatness of function
- User-specified parameter e defines “tube”
- Support vectors: points on or outside tube

SVM Regression

Regression: $f(x) = wx + b$



Smola and Scholkopf, 2003

Hard Margin Formulation

$$\text{minimize} \quad \frac{1}{2}\|w\|^2$$

$$\text{subject to} \quad \begin{cases} y_i - \langle w, x_i \rangle - b & \leq \quad \varepsilon \\ \langle w, x_i \rangle + b - y_i & \leq \quad \varepsilon \end{cases}$$

Smola and Scholkopf, 2003

Soft Margin Formulation

$$\text{minimize} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{\ell}(\xi_i + \xi_i^*)$$

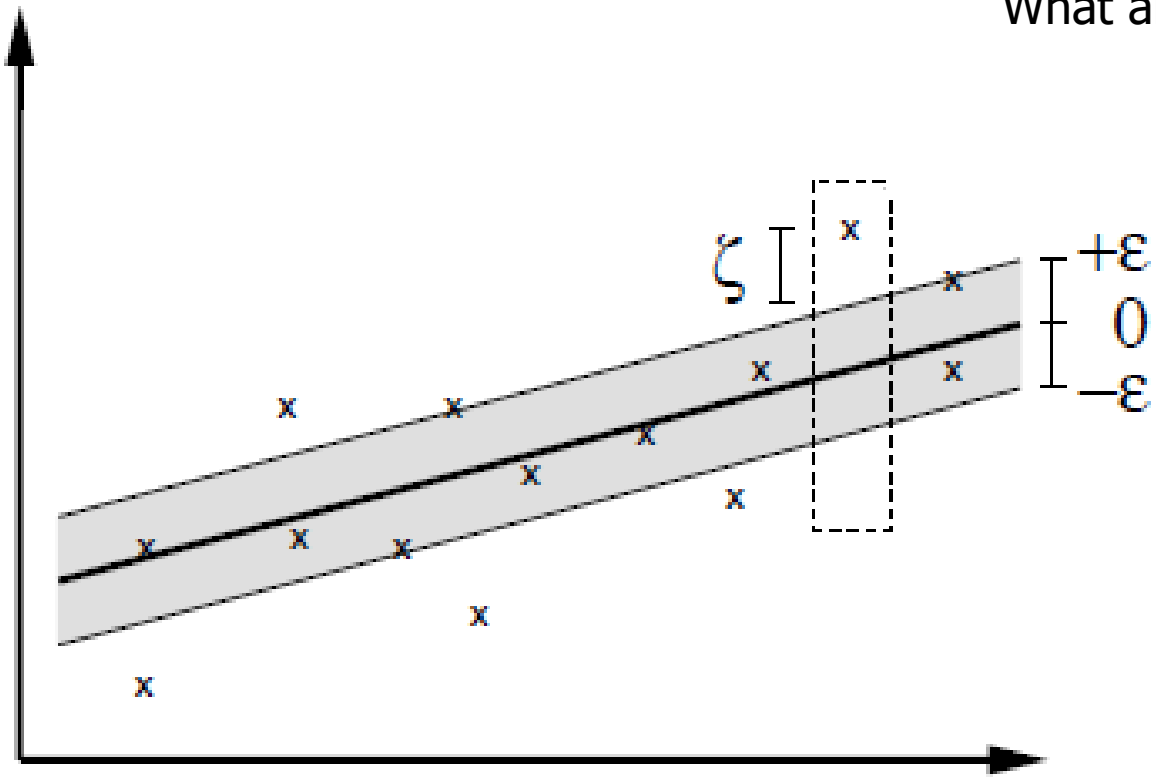
$$\text{subject to} \quad \begin{cases} y_i - \langle w, x_i\rangle - b & \leq & \varepsilon + \xi_i \\ \langle w, x_i\rangle + b - y_i & \leq & \varepsilon + \xi_i^* \\ \xi_i, \xi_i^* & \geq & 0 \end{cases}$$

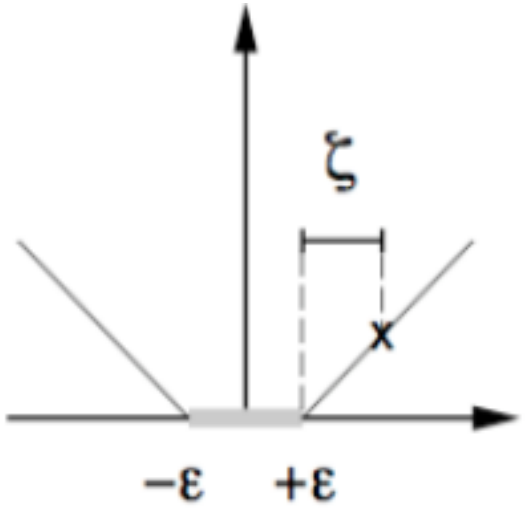
The constant $C > 0$ determines the trade-off between the flatness of f and the amount up to which deviations larger than ε are tolerated.

Smola and Scholkopf, 2003

Support Vectors and Weights

Which data points are support vectors?
What are their weights?





Smola and Scholkopf, 2003